

NIST Special Publication 1017-2
Sixth Edition

**Smokeview, A Tool for Visualizing
Fire Dynamics Simulation Data
Volume II: Technical Reference Guide**

Glenn P. Forney

NIST Special Publication 1017-2
Sixth Edition

**Smokeview, A Tool for Visualizing
Fire Dynamics Simulation Data
Volume II: Technical Reference Guide**

Glenn P. Forney
*Fire Research Division
Engineering Laboratory*

May 17, 2026
Revision: SMV-6.11.0-0-ga0be6bd



U.S. Department of Commerce
Howard Lutnick, Secretary

National Institute of Standards and Technology
Craig Burkhardt, Acting NIST Director and Acting Undersecretary of Commerce for Standards and Technology

Certain commercial entities, equipment, or materials may be identified in this document in order to describe an experimental procedure or concept adequately. Such identification is not intended to imply recommendation or endorsement by the National Institute of Standards and Technology, nor is it intended to imply that the entities, materials, or equipment are necessarily the best available for the purpose.

National Institute of Standards and Technology Special Publication 1017-2
Natl. Inst. Stand. Technol. Spec. Publ. 1017-2, 74 pages (June 2016)
CODEN: NSPUE2

Preface

This guide is part of a three volume set of companion documents describing Smokeview. Volume I, the Smokeview User's Guide [1], describes how to use Smokeview. Volume II, the Smokeview Technical Reference Guide [2], gives technical details of how the visualizations are performed. Volume III, the Smokeview Verification Guide [3] presents example cases verifying the various visualization capabilities of Smokeview. Details on the use and technical background of the Fire Dynamics Simulator is contained in the FDS User's [4] and Technical reference guide [5] respectively. This guide is Volume II the Smokeview Technical Reference guide.

Smokeview is a software tool designed to visualize numerical calculations generated by the Fire Dynamics Simulator (FDS), a computational fluid dynamics (CFD) model of fire-driven fluid flow and CFAST a two-zone fire model used to calculate the evolving distribution of smoke, fire gases and temperature. This report documents some of the algorithms Smokeview uses to visualize fire dynamics data giving some of the technical and programming details. Details on the use of Smokeview may be found in the Smokeview User's Guide. Smokeview uses the 3D graphics library OpenGL to visualizing fire and smoke data. This library is used to specify the location, color and lighting of objects residing within a *3D world* defined by FDS. In the context of FDS, these objects may be used to represent geometry (such as blockages) or to visualize data. Smokeview presents fire modeling data using visualization techniques such as tracer particles, 2D shaded contours, iso-surfaces or flow vectors. Soot data or smoke is visualized using a variation of the 2D shaded contour technique where transparency rather than color is used to represent the opacity or optical thickness of smoke. The details used to implement various techniques for visualizing smoke are discussed.

About the Author

Glenn Forney is a computer scientist at the Engineering Laboratory of NIST. He received a bachelor of science degree in mathematics from Salisbury State College and a master of science and a doctorate in mathematics from Clemson University. He joined NIST in 1986 (then the National Bureau of Standards) and has since worked on developing tools that provide a better understanding of fire phenomena, most notably Smokeview, a software tool for visualizing Fire Dynamics Simulator data.

Disclaimer

The US Department of Commerce makes no warranty, expressed or implied, to users of Smokeview, and accepts no responsibility for its use. Users of Smokeview assume sole responsibility under Federal law for determining the appropriateness of its use in any particular application; for any conclusions drawn from the results of its use; and for any actions taken or not taken as a result of analysis performed using this tools.

Smokeview and the companion program FDS is intended for use only by those competent in the fields of fluid dynamics, thermodynamics, combustion, and heat transfer, and is intended only to supplement the informed judgment of the qualified user. These software packages may or may not have predictive capability when applied to a specific set of factual circumstances. Lack of accurate predictions could lead to erroneous conclusions with regard to fire safety. All results should be evaluated by an informed user.

Throughout this document, the mention of computer hardware or commercial software does not constitute endorsement by NIST, nor does it indicate that the products are necessarily those best suited for the intended purpose.

Contents

Preface	i
About the Author	iii
Disclaimer	v
1 Introduction	1
1.1 Basic Description of Smokeview	1
1.2 Version History	2
1.3 Model Development	2
1.4 Capabilities	3
1.5 Overview	3
2 Preliminaries	5
2.1 Defining Objects	5
2.1.1 Setting the Scene	7
2.1.2 Projections	7
2.1.3 Stereo Projections	7
2.1.4 Viewports	9
2.2 Coloring, Shading and Blending Objects	9
2.2.1 Color	9
2.2.2 Shading	10
2.2.3 Blending	10
2.3 Motion	15
2.3.1 Implementation	16
3 Visualizing Data	19
3.1 Coloring Data	19
3.1.1 Determining Data Bounds	20
3.1.2 Converting data to a color	20
3.1.3 Interpolating Colors	20
3.2 2D Contours	24
3.2.1 Line contours	24
3.2.2 Banded contours	24
3.3 3D Contours - Isosurfaces	28
3.4 Particle Systems	34
3.4.1 Massless Particles	34
3.5 Computing and Visualizing Fractional Effective Dose data	34
3.5.1 FED example	35

4	Visualizing Smoke and Fire	41
4.1	Introduction	41
4.2	Radiation Transport Equation	42
4.2.1	Approximating the Radiation Transport Equation	43
4.2.2	Discretizing the Radiation Transport Equation	44
4.2.3	Splitting the Radiation Transport Equation	46
4.3	Slice Rendering	47
4.3.1	Computing Color	48
4.3.2	Computing Opacity	48
4.3.3	Adjusting Opacity	49
4.3.4	Orienting Slice Planes	50
4.3.5	Compressing Slice Plane Data	51
4.3.6	A Limitation of the Slice Solution Method	51
4.4	Volume Rendering	52
4.4.1	Implementation	52
4.5	Future Work	53
5	Future Work	61
5.1	Visualize Cases more Realistically	61
5.2	Fire Computations in Smokeview	61
5.3	Visualize Larger Cases	62
5.4	Tools and Techniques	62
	References	65
	Appendices	66
A	Smokeview Program Structure	67
B	Interfacing OpenGL with the Host Operating System	71
B.1	Buffers	71
B.2	Initialization and Callback Routines	71
C	Rendering Smokeview Images	73

List of Figures

2.1	Right hand rule used by Smokeview for specifying a 3D vertex locations	5
2.2	Points, lines and a shaded triangle drawn using OpenGL	6
2.3	Example view frustum used to convert 3D scenes to 2D screen viewport	8
2.4	View frustums for stereo pairs	8
2.5	Examples of several viewports in a typical Smokeview scene	9
2.6	The FDS townhouse case drawn using flat and smooth shading	11
2.7	Two spheres drawn showing the effect of using averaged normals	12
2.8	A slice file drawn transparently mixes slice colors with those in the background	14
2.9	Diagram relating the vector $u \times v$ and the angle θ with vectors u, v	17
3.1	1D colorbar and 3D color cube	21
3.2	Slice file snapshots illustrating old and new method for coloring data	23
3.3	Color interpolation examples	25
3.4	2D line contour canonical forms	26
3.5	2D band contour canonical forms	26
3.6	Snapshot of an isosurface of temperature at 100 °C (212 °F)	30
3.7	Snapshot of an isosurface of temperature at 100 °C (212 °F)	31
3.8	3D isosurface canonical forms	32
3.9	Example of triangle decimation	33
3.10	Setup for determining the slope of a smooth curve passing through three points	36
3.11	Setup for determining isosurface opacity as a function of orientation	37
3.12	Isosurface with variable opacity	38
3.13	Plume flow visualized using particles and particle streaks	39
3.14	Plot of FED vs. time for ambient and smoke filled rooms	40
4.1	Diagram illustrating components of the radiation transport equation	43
4.2	Setup for discretizing the equations used to model radiance	44
4.3	Solutions to the RTE on two sub-intervals	46
4.4	Example colormap used for converting temperature or hrrpuv values to color	48
4.5	Light emitted from the background is obscured by intervening smoke	49
4.6	Diagram illustrating the adjustment required to the opaqueness parameter	54
4.7	Smoke plume visualized using several vertical parallel partially transparent planes	55
4.8	Realistic visualization of a townhouse kitchen fire simulated using FDS	56
4.9	Realistic visualization of a townhouse kitchen fire simulated using FDS viewed along the X axis.	57
4.10	Slice plane orientation chosen to be <i>most perpendicular</i> to the line of sight	57
4.11	Diagram illustrating the angle between sight direction and the slice plane normal vector	58
4.12	Opacity and color is computed by solving a simplified form of the radiation transport equation	58

4.13	Volume rendered smoke plume shown from several points of view	59
A.1	Smokeview external library usage	68
A.2	Smokeview program structure	69

Chapter 1

Introduction

1.1 Basic Description of Smokeview

Smokeview is a software tool designed to visualize numerical predictions generated by the Fire Dynamics Simulator (FDS), a computational fluid dynamics (CFD) model of fire-driven fluid flow [4, 6] and CFAST a two-zone fire model [7] used to calculate the evolving distribution of smoke, fire gases and temperature. Most of Smokeview is written in the C programming language using the 3D graphics library, OpenGL [8], for implementing visualization algorithms and GLUT [9] for interacting with both the user and the operating system. More specifically, OpenGL is used to specify the location, color and lighting of objects residing within a *3D world* defined by FDS. In the context of FDS, these objects may be used to represent geometry (such as blockages) or to visualize data. Smokeview uses these underlying techniques as building blocks to visualize data such as tracer particles, 2D shaded contours or 3D level iso-surfaces. Soot data or smoke may also be visualized using a variation of a 2D shaded contour, where transparency rather than color is used to represent the opacity or optical thickness of smoke. This report describes these algorithms giving some of the technical and programmatic details.

Smokeview is comprised of approximately 175 000 lines of code. Most of it is written in C [10]. A small but important part is written in Fortran 90 [11]. This part is used to read data generated by FDS. Additional software libraries (about 250 000 lines of code) are used for implementing dialogs, rendering images and decompressing data files. The use of portable libraries allows Smokeview to run on many platforms including Windows, Linux and OS X (for the Macintosh).

The fundamental purpose of visualization is to gain insight into the phenomena being studied. There is no one best method for visualizing data. Each visualization technique highlights a different aspect of the data. Smokeview visualizes fire dynamics data, typically results from the Fire Dynamics Simulator. This data takes many forms. Some data is static, whereas other data evolves with time. Some data represents geometric objects whereas other data represents the solution to the flow equations solved by FDS or a zone fire model such as CFAST. Smokeview displays fire dynamics data allowing quantitative assessment to be performed using visualization techniques such as animated tracer particles that follow the flow, animated shaded 2D and 3D contours that display flow quantities and animated flow vectors that display flow quantities and direction. Smokeview also visualizes smoke realistically by converting soot density to smoke opacity, displaying smoke as it would actually appear. Each of these visualization techniques highlight different aspects of the underlying flow phenomena.

For details on the use of Smokeview, the reader is advised to read the Smokeview User's Guide [1]. For details on how Smokeview is verified, see the Smokeview Verification Guide [3]. For details on setting up and running FDS cases, read the FDS User's Guide [4]. For details on the theory and algorithms implemented in FDS read the FDS Technical Guide [6].

1.2 Version History

Beginning in the early 1980s and continuing into the 1990s, Howard Baum and Ronald Rehm of NIST developed the basic flow solver that evolved into the Fire Dynamics Simulator which was publicly released in 2000[4]. Their solution technique, known as Large Eddy Simulation or LES, captures numerically very complicated fire plume dynamics. Unfortunately, the power of the methodology could not be appreciated without an effective way to view the calculation results. Early attempts to visualize the calculation results consisted of nothing more than little particles swirling about in a box. This was useful to the model developers, but hardly to anyone else. It just did not look like a fire.

Smokeyview was written to address this problem. It is an advanced scientific visualization tool whose drawing algorithms are based on physics, not just a tool for drawing pretty pictures. Version 1 of Smokeyview was publicly released in February 2000, version 2 in December 2001, version 3 in November 2002, version 4 in July 2004 and version 5 in September 2007. The present version of Smokeyview is 6, officially released in June 2012.

Along with particle tracking as performed before, it visualized fire flow data by coloring and animating fire/smoke flow making it much easier to interpret FDS simulation results. Immediately after September 11, 2001, work began on both FDS and Smokeyview to enable them to model and visualize much larger problems. As a result, fire scenarios with several million grid cells can now be modeled and visualized using a cluster of computers.

The next big step in Smokeyview's development was the implementation of an algorithm for realistically visualizing smoke. The line between FDS which performs smoke flow computations and Smokeyview which performs smoke flow visualization became blurred as Smokeyview now performs physics-based computations (Beer's law) in order to visualize the smoke. The present algorithm for visualizing smoke only considers the effects of absorption - how much an object is obscured by smoke. Future work involves modeling the effects of scattering - how the interaction between light and smoke effects the visualization and coloring the fire more realistically based on physics principles such as the blackbody temperature curve. The challenge is to represent color in an appropriate manner, considering the complexities associated with perception. For example, one complication is that often the color that one perceives depends on what was just seen.

1.3 Model Development

Currently, Smokeyview is maintained by the Engineering Laboratory (EL) of the National Institute of Standards and Technology. The FDS-Smokeyview development team uses git for version control and the website github.com to assist open source software development by providing a repository for source code, revision control, program distribution, bug tracking, and various other very useful services.

The Smokeyview manuals are typeset using $\text{\LaTeX}$, specifically, PDF $\text{\LaTeX}$. The $\text{\LaTeX}$ files are essentially text files that are under GIT version control. The figures are either in the form of PDF or jpeg files, depending on whether they are vector or raster format. There are a variety of $\text{\LaTeX}$ packages available, including MiKTeX, a Windows version. The FDS-Smokeyview developers edit the manuals as part of the day to day upkeep of the model. Different editions of the manuals are distinguished by date.

Changes made to Smokeyview are tracked using revision control software. Not every change results in a change to the version number. The version number changes after it is judged that the source code changes warrant a version number change. For minor changes and bug fixes, incremental versions are released, referenced according to fractions of the integer version number. For example, version 5.1.4 would be a maintenance release of feature version 5.1, which in turn is an update within the major application release referred to as Smokeyview 5. This happens every few weeks. A minor release from 5.2 to 5.3, for example,

might happen only a few times a year, when significant improvements have been made to the visualizations.

A suite of simple verification visualizations are routinely run to ensure that the daily changes have not altered any of the important visualization algorithms. The Smokeview verification suite is documented in Ref [3].

1.4 Capabilities

Smokeview is a program designed to visualize numerical calculations generated by the Fire Dynamics Simulator. Smokeview visualizes both dynamic and static data. Dynamic data is visualized by animating particle flow (showing location and *values* of tracer particles), 2D contour slices (both within the domain and on solid surfaces) and 3D iso surfaces. 2D contour slices can also be drawn with colored vectors that use velocity data to show flow direction, speed and value. Static data is visualized similarly by drawing 2D contours, vector plots and 3D level surfaces. Smokeview features in more detail include:

Particle Animations Lagrangian or moving particles can be used to visualize the flow field. Often these particles represent smoke or water droplets.

Color Contours Animated 2D shaded color contour plots are used to visualize gas phase information, such as temperature or density.

Animated Flow Vectors Flow vector animations, though similar to color contour animations (the vector colors are the same as the corresponding contour colors), are better than solid contour animations at highlighting flow features.

Animated Isosurfaces Isosurface or 3D level surface animations may be used to represent flame boundaries, layer interfaces and various other gas phase variables.

Realistic Smoke Smoke, fire and sprinkler spray are displayed realistically using a series of partially transparent planes.

This report describes some of the technical details involved in implementing these capabilities.

1.5 Overview

Smoke and other attributes of fire are visualized by Smokeview using both quantitative and realistic techniques. Smokeview uses quantitative display techniques such as 2D and 3D contouring. Realistic display of data refers to the intent of presenting the data in a form as it would actually appear. The first part of this document presents some of the details involved in using these techniques for displaying FDS generated data.

To implement these techniques, Smokeview uses various tools and techniques such as color, lighting, motion and transformation. These basic building blocks, discussed in Appendix 2, are used by each of the techniques Smokeview uses for visualizing data, in particular, smoke visualization. A more thorough treatment may be found in Martz's OpenGL Distilled [12], the OpenGL Programming Guide (Red book) [8] or the SuperBible [13]. Appendix A gives some of the programmatic details in particular giving the structure and naming conventions of some of the program source files and how the external libraries are organized and used. Appendix B gives the details of how the OpenGL components used by Smokeview interact with the host operating system. Finally, several miscellaneous topics such as image rendering and data compression are detailed.

Chapter 2

Preliminaries

2.1 Defining Objects

Smokeview defines object locations in terms of a right handed coordinate system with the x axis pointing to the right, the y axis pointing to the back and the z axis pointing up (see Fig. 2.1). Coordinate values as defined in an FDS input file for objects such as blockages, vents, etc. are transformed internally within Smokeview to lie between 0.0 and 1.0. All Smokeview objects are drawn by grouping appropriately *OpenGL* vertices.

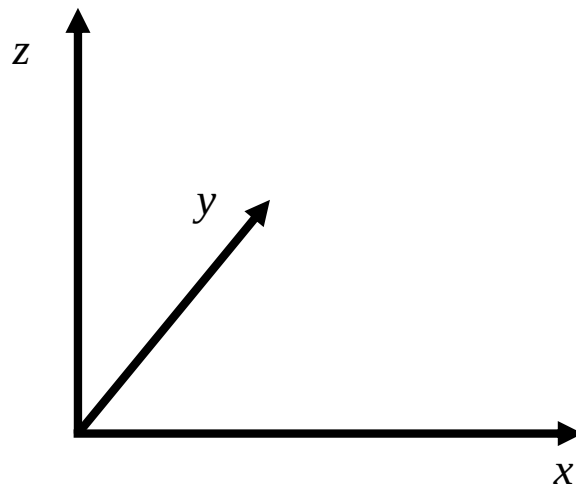


Figure 2.1: Right hand rule used by Smokeview for specifying a 3D vertex locations.

A vertex in OpenGL has the same meaning as in geometry, a location. Vertices are specified in Smokeview using either `glVertex3f(x,y,z)` or `glVertex3fv(xv)`. The suffix for `glVertex` indicates the number and type of data value to be passed. For example, `3f` is used when passing 3 scalar floating points. The suffix `3fv` is used to when passing a pointer (or equivalently a memory address) to 3 floating point values.

To illustrate, suppose that `xyz` is a floating point array of size 3 containing a 3D coordinate. One could then use either of the two OpenGL calls:

```
glVertex3fv(xyz);  
glVertex3f(xyz[0], xyz[1], xyz[2]);
```

to represent the vertex location.

Groups of vertices may be *built up* to form more complex geometric objects. Several objects are illustrated in Fig. 2.2. They are formed by grouping vertices together and surrounding them with calls to `glBegin()` and `glEnd()`. The argument passed to `glBegin()` determines which higher level object is drawn. To draw a shaded triangle one would use

```
glBegin(GL_TRIANGLE);  
glVertex3f(0.0,0.0,0.0);  
glVertex3f(0.0,1.0,0.0);  
glVertex3f(1.0,0.0,0.0);  
glEnd();
```

Similarly, to draw points or to connect the vertices with lines (also shown in Fig. 2.2) one would replace `GL_TRIANGLE` above with `GL_POINTS` or `GL_LINES` respectively.

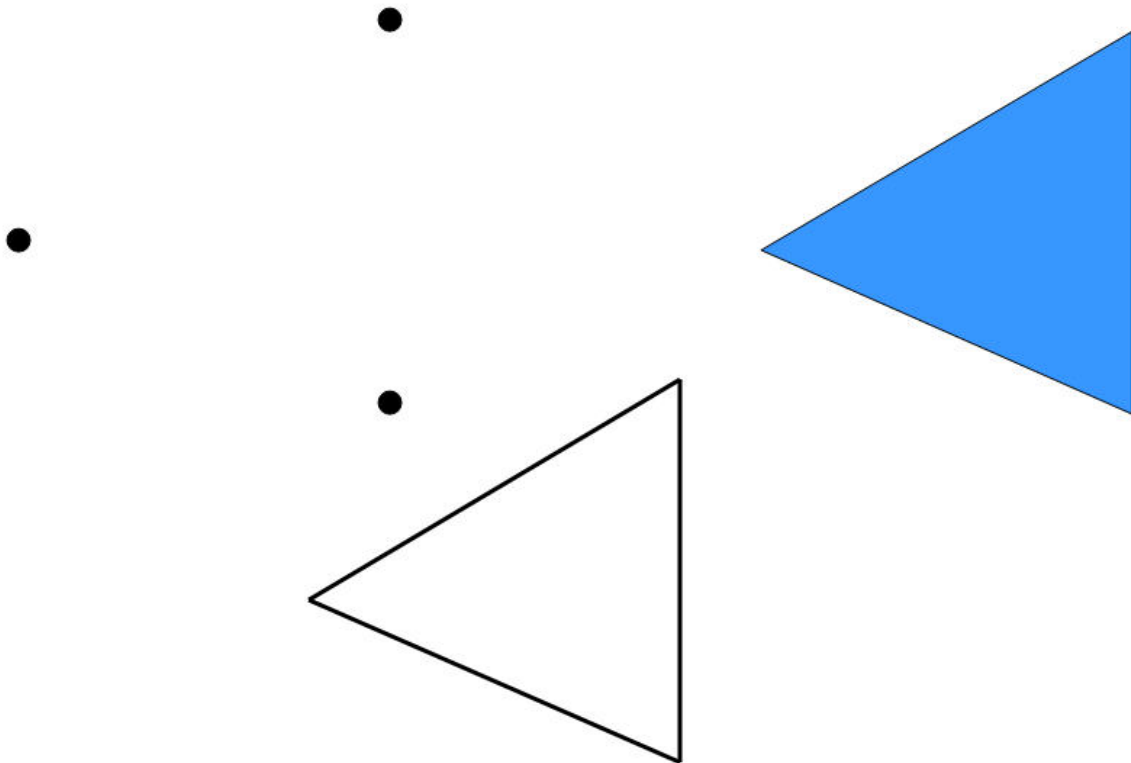


Figure 2.2: Points, lines and a shaded triangle drawn using OpenGL. Vertices are defined using `glVertex*` and the particular shapes are generated by passing `GL_POINTS`, `GL_LINES`, and `GL_TRIANGLE` to `glBegin()`

The triangle is the fundamental construct Smokeview uses to visualize objects. To be useful though, these objects need to be colored, moved and projected onto a 2D terminal screen. These topics are discussed in the following sections.

2.1.1 Setting the Scene

3D objects must be flattened or converted to 2D in order to display them on a computer screen. This occurs in two steps. The first step is called a projection. Smokeview uses three kinds of projections: perspective, orthographic and stereo. The second step takes whatever projection that has been applied and maps the resulting flattened geometry onto a subset of the computer screen. This window subset, usually a rectangle, is called a viewport. All viewports in Smokeview are rectangular.

2.1.2 Projections

Projections are used to flatten the 3D scene onto a two dimensional plane. Two common projections are orthographic and perspective. An orthographic projection is size preserving. Objects in the foreground take up the same amount of screen space as objects in the background. This projection is sometimes called a parallel projection because parallel lines in the 3D scene remain parallel when drawn on the screen.

A perspective projection creates an illusion of depth by causing objects in the background to take up less screen space than the same sized object drawn in the foreground. Both projection methods are available in Smokeview.

Smokeview uses `glFrustum` to perform perspective projections

```
glFrustum(  
    (double) fleft, (double) fright,  
    (double) fdown, (double) fup,  
    (double) fnear, (double) ffar);
```

and `glOrtho` to perform orthographic projections

```
glOrtho(  
    (double) fleft, (double) fright,  
    (double) fdown, (double) fup,  
    (double) fnear, (double) ffar);
```

where `fleft`, `fright`, `fdown`, `fup`, `fnear`, `ffar` are six clipping planes bounding the view frustum (truncated pyramid) for the perspective projection or the box in the orthographic projection. Drawing does not occur outside of the 3D region defined by these 6 clipping planes. An additional six clipping planes parallel to the x, y and z axes may be activated in Smokeview (using the clipping dialog box) to hide geometry making it easier to see interior objects or visualizations (hence the term clipping plane). OpenGL allows one to define clipping planes along arbitrarily oriented planes.

2.1.3 Stereo Projections

A stereo projection is simply a perspective applied twice making adjustments to simulate the viewpoint as seen for an observer's left and right eye. The two resulting versions of the scene can then be drawn in succession using shuttered glasses synced with the monitor to render the stereo effect. Alternatively, left and right versions of the scene may be drawn at the same time using stereo viewers to reveal the stereo effect. Figure 2.4 illustrates the two view frustums used for a stereo projection.

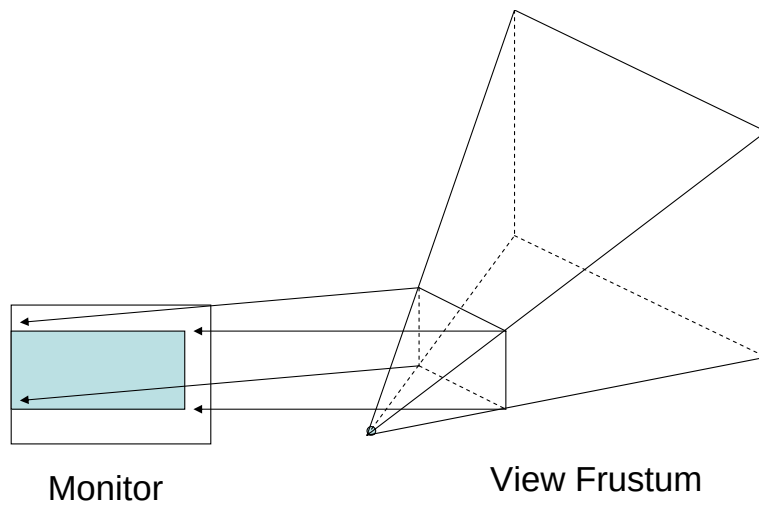


Figure 2.3: Example view frustum used to convert 3D scenes to 2D screen viewport.

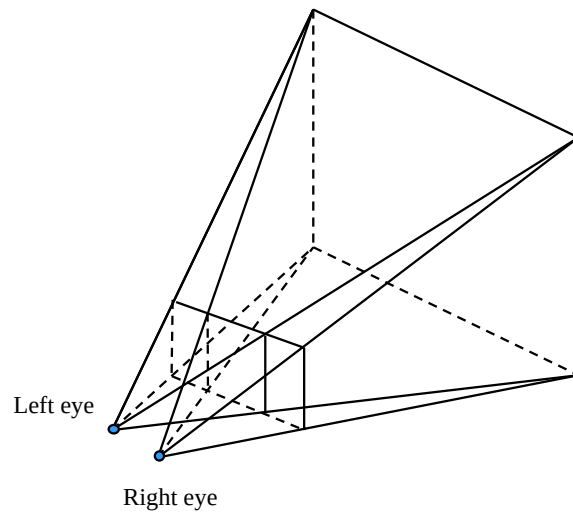


Figure 2.4: View frustums for stereo pairs.

2.1.4 Viewports

A viewport is the particular portion of the screen where the drawing occurs. Smokeview defines separate viewports for drawing the title, time bar, color bar and the 3D scene. Figure 2.5 illustrates the relationship between the 3D scene and the 2D screen giving several viewport examples used by Smokeview.

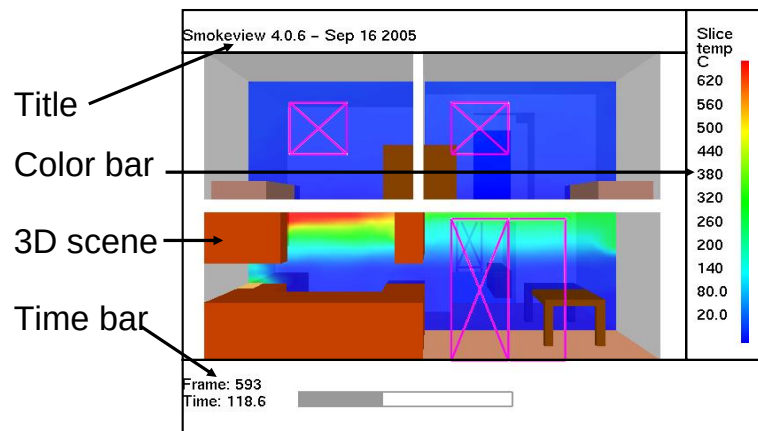


Figure 2.5: Examples of several viewports in a typical Smokeview scene.

Smokeview uses `glViewport()` to establish a window viewport. In particular, the call

```
glViewport(left,down,width,height);
```

is used to set up the portion of the screen where 3D drawing occurs where `left` and `down` are the bottom left screen coordinates of the viewport and `width` and `height` are the width and height of the viewport.

2.2 Coloring, Shading and Blending Objects

Color, shading and blending are three aspects of the same process used to draw objects residing in a three dimensional environment. Smokeview uses color to display and more importantly to distinguish between two or more geometric or data elements. Shading adds a further depth cue (besides perspective projections discussed earlier) to present the illusion of 3 dimensions. Using a virtual light source, colors are changed subtly based upon the relative orientation of the light source and the surface being drawn. Portions of the surface oriented away from the light are drawn darker than portions oriented towards the light. Blending is the process of combining the colors of two or more objects in such a way that objects in the background may be seen through semi-transparent objects drawn in the foreground. Together, color, shading and blending add to the illusion that the image drawn on a two dimensional terminal screen is three dimensional.

2.2.1 Color

Color in OpenGL is defined using four not three components. Besides the expected components of red, green and blue, OpenGL uses a component designated as *alpha* to represent opaqueness. Each component ranges from 0.0 to 1.0. An alpha component of 0.0 indicates a color that is completely transparent while an alpha component of 1.0 represents a color that is completely opaque.

Colors are specified using the OpenGL routine `glColorXXX()`. As with `glVertexXXX()`, `XXX` is replaced by a suffix used to indicate the number and type of argument passed. For the most part, Smokeview uses `glColor3f`, `glColor4f`, `glColor3fv` and `glColor4fv`. As before, the 3 or 4 indicates the number of data values used. If 3 is specified in the call then it is tacitly assumed that the fourth component, `alpha`, has value 1.0. In other words, the color is opaque. The `f` parameter indicates that floating point values are used and the `v` parameter indicates that the variable passed is a pointer to the actual data to be referenced.

2.2.2 Shading

OpenGL uses two shading models for drawing objects, flat and Gouraud. These models are specified using `glShadeModel(GL_FLAT)` and `glShadeModel(GL_SMOOTH)` respectively. Gouraud shading is also referred to as smooth shading. Flat shading assumes that objects are drawn in an environment with uniform lighting - light surrounds an object equally in all directions. Smooth lighting on the other hand assumes that light comes from a particular direction. This causes subtle changes in color to occur across an object's surface. Figure 2.6 shows examples of FDS blockages (the standard townhouse scenario) drawn using flat and smooth shading. Flat shading diminishes the three dimensionality of the scene.

A normal vector, another vertex attribute, is required to implement a smooth lighting scheme. A normal vector points in a direction perpendicular to the surface at the vertex. OpenGL uses this information to estimate the fraction of light from the light source reflected off of the given surface and intercepted by the observer. This is similar to a configuration factor calculation performed in fire modeling. The amount of light perceived by the observer depends on the relative orientation of the light source, the object (as specified by the location and normal vector of each vertex) and the observer.

For planar surfaces, the same normal vector is applied to each vertex defining the surface. For curved surfaces, normal vectors are determined using an average of the normal directions of faces surrounding the vertex. If normals are not averaged then the discontinuity in slope going from one face or triangle to another will result in a faceted or gem-like appearance. Figure 2.7 shows examples of drawing using non-averaged and averaged normals.

The Gouraud method for shading then determines a vertex color using the angle between the light source direction and the vertex normal vector. The color of the object being shaded is then determined by interpolating these colors.

Smokeview uses smooth shading or lighting to draw blockages and iso-surfaces. Particle, slice and boundary files are drawn without shading as are 3D smoke and Plot3D files.

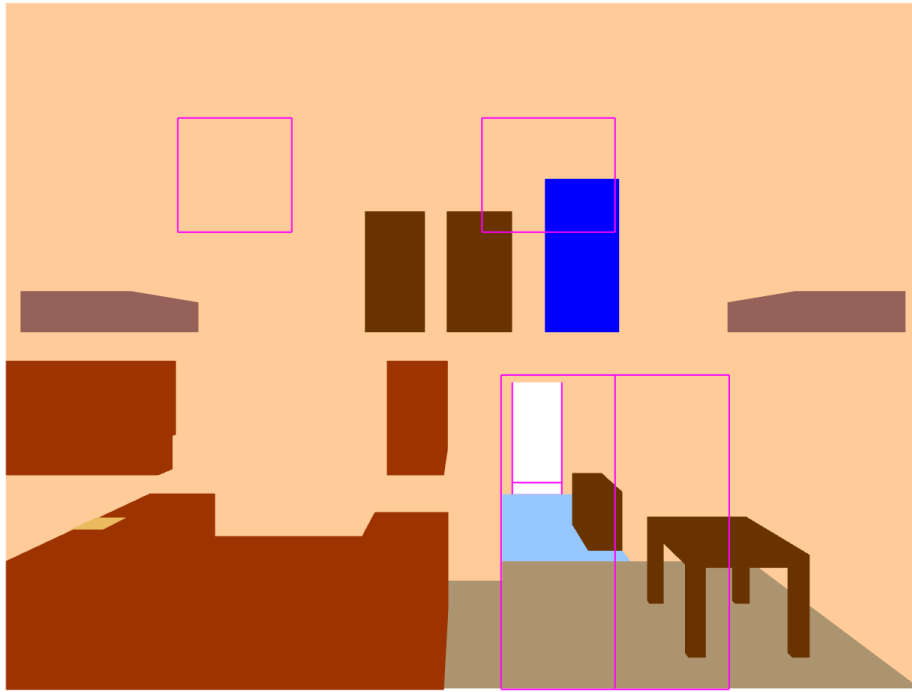
2.2.3 Blending

Smokeview using OpenGL draws semi-transparent objects by combining or blending the color of the object currently being drawn with the color in the current background buffer.¹ The blending fraction is determined from the alpha color component of the object currently being drawn. A small alpha results in a small contribution from the currently drawn object color while a large alpha results in a large contribution. More precisely:

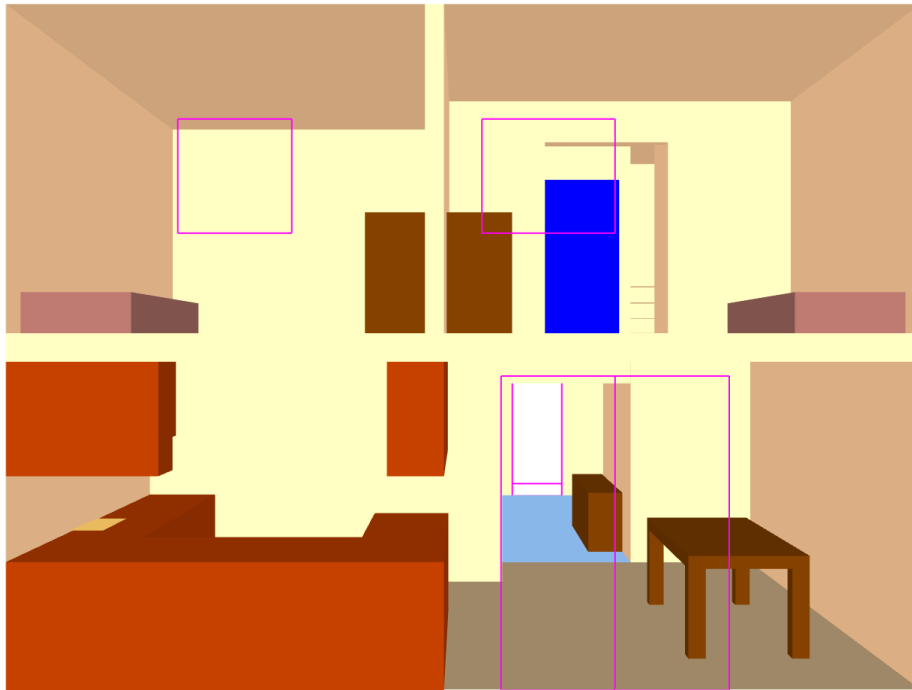
$$\text{updated background color} = \alpha \times \text{fragment color} + (1 - \alpha) \times \text{original background color} \quad (2.1)$$

This particular blending model is the most common (there are others) and is activated in Smokeview using the OpenGL call

¹The background buffer is updated as each object is drawn.

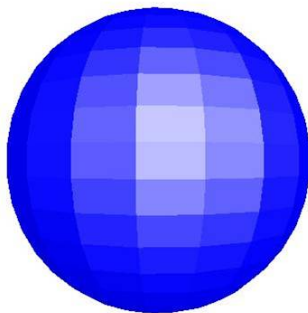
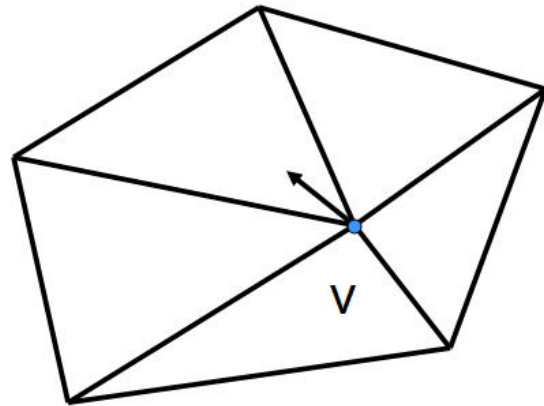
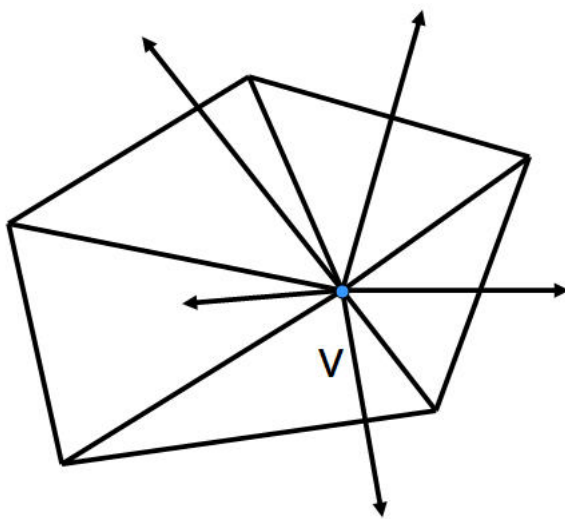


flat shading

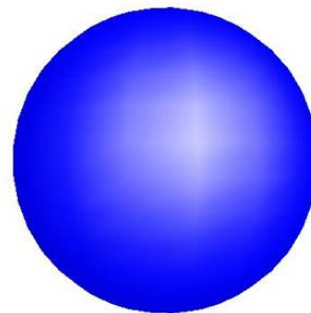


smooth (Gouraud) shading

Figure 2.6: The FDS townhouse case drawn using flat and smooth shading. All blockage surfaces have identical colors when drawn with flat shading. When drawn with smooth shading, blockage colors change. Surfaces are darker when not in direct view of the light source adding to a sense of depth.



separate normals $\rightarrow$ faceted drawing



averaged normals $\rightarrow$ smooth drawing

Figure 2.7: Two spheres drawn showing the effect of using averaged normals. Using non-averaged normals results in a faceted or gem-like appearance.

```
glBlendFunc(GL_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

Choosing an alpha less than one allows one to *see through* objects. Smokeview uses this feature to implement partially transparent blockages and 2D animated slices. Slice files illustrated in Fig. 2.8 are drawn using transparency. Smokeview also implements data chopping or hiding using blending or transparency. Data to be hidden is assigned an alpha value of 0.0 causing it to be completely transparent.

Complications arise, however, because this blending model is not commutative. The final color the user sees depends on the order in which the intermediate colors are drawn. To demonstrate this simply, consider two objects, one opaque and one semi-transparent. An opaque object drawn second and in front of a semi-transparent object will totally obscure the semi-transparent object. A semi-transparent object drawn second and in front of an opaque object will blend with but not obscure the opaque object drawn previously.

To be more precise, consider two objects with colors c_1 and c_2 and alpha values α_1 and α_2 . Let the background color be denoted b_0 . Then blending object 1 with the background results in a new background color, b_1 , given by

$$b_1 = (1 - \alpha_1)b_0 + \alpha_1 c_1 \quad (2.2)$$

Blending the color c_2 with b_1 results in a new background color, b_2 , given by

$$b_2 = (1 - \alpha_2)b_1 + \alpha_2 c_2 \quad (2.3)$$

$$= (1 - \alpha_2)((1 - \alpha_1)b_0 + \alpha_1 c_1) + \alpha_2 c_2 \quad (2.4)$$

$$= (1 - \alpha_2)(1 - \alpha_1)b_0 + \alpha_1 c_1 + \alpha_2 c_2 - \alpha_2 \alpha_1 c_1 \quad (2.5)$$

Now blend colors c_1 and c_2 to the original background in the opposite order. Blending the color c_2 to with the background color b_0 results in a new background color, $\hat{b}_1$, given by

$$\hat{b}_1 = (1 - \alpha_2)b_0 + \alpha_2 c_2 \quad (2.6)$$

Blending c_1 to the interim background color $\hat{b}_1$ results in a new background color, $\hat{b}_2$, given by

$$\hat{b}_2 = (1 - \alpha_1)\hat{b}_1 + \alpha_1 c_1 \quad (2.7)$$

$$\hat{b}_2 = (1 - \alpha_1)((1 - \alpha_2)b_0 + \alpha_2 c_2) + \alpha_1 c_1 \quad (2.8)$$

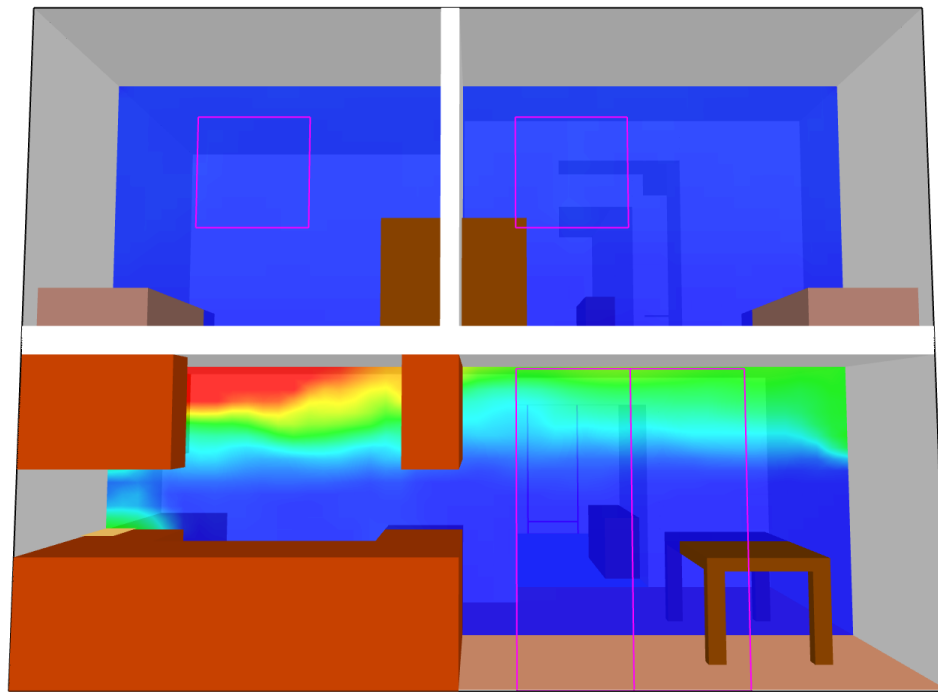
$$\hat{b}_2 = (1 - \alpha_1)(1 - \alpha_2)b_0 + \alpha_2 c_2 + \alpha_1 c_1 - \alpha_1 \alpha_2 c_2 \quad (2.9)$$

In general, $b_2 - \hat{b}_2 = \alpha_1 \alpha_2 (c_2 - c_1) \neq 0$, unless $c_1 = c_2$. Of course $b_2 = \hat{b}_2 = 0$ if $\alpha_1 = 0$ or $\alpha_2 = 0$ but this is a trivial case in which one of the two fragments is completely transparent.

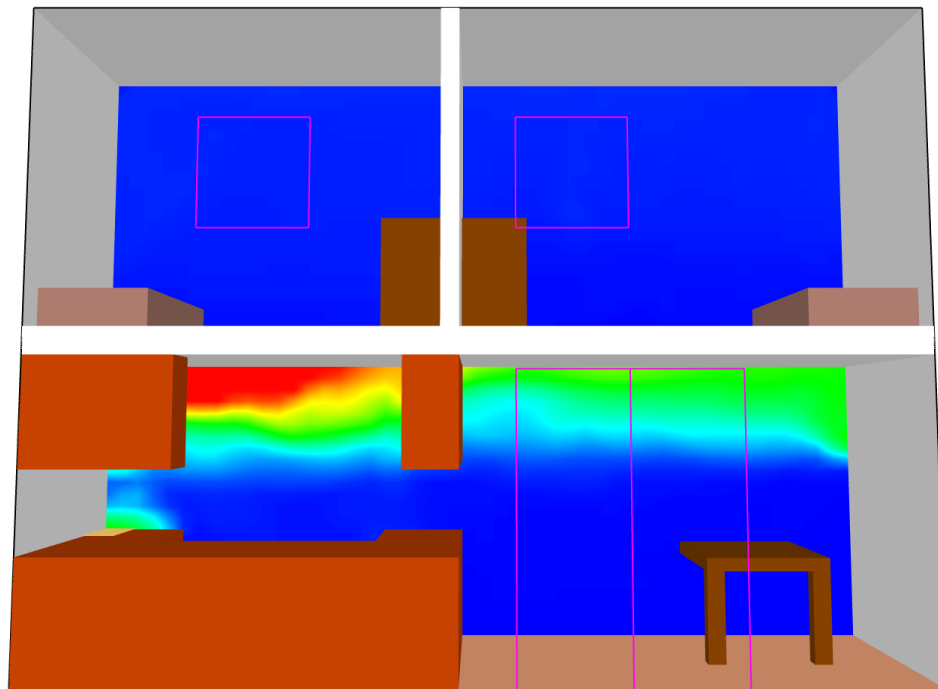
The application to Smokeview is that since all smoke is drawn with the same color, smoke slice plane drawing is order independent. However, whenever fire is visualized along with smoke, multiple colors may exist in a given plane (black for smoke, orange for fire). In this case, the order that the planes are drawn becomes important.

In order then to prevent inconsistent drawing for the general case, opaque objects are drawn first then partially transparent objects are drawn next from back to front (from the point of view of the observer). Otherwise transparent objects may appear blended in front of objects when they should in fact be obscured. Of significance in Smokeview, is that slice plane ordering is not important when drawing 3D smoke since all smoke is drawn with the same color (but different opacities). However, order is important when drawing smoke and fire (two different colors) or if considering more elaborate lighting algorithms where in general every vertex may have a different color (due to lighting effects).

When lighting is not applied, colors within a triangle are determined in two steps using bi-linear interpolation. First, colors along a triangle edge are linearly interpolated using the two colors of the vertices



partially transparent slice plane



opaque slice plane

Figure 2.8: A slice file drawn transparently mixes slice colors with those in the background. When drawn opaquely, any portion of the scene behind the slice file is hidden.

bounding the edge. Second, the interior colors are determined along a horizontal scan line again using linear interpolation using colors previously interpolated on the triangle edge.

Flat shaded triangles may be drawn more efficiently but are not effective at visualizing a 3D effect. More sophisticated shading techniques are required and are discussed next.

2.3 Motion

Previous sections discussed how appearance is important in visualization. This section discusses how motion may be used to gain insight into fire phenomena. Motion may be thought of in two separate but equivalent ways - keeping the scene fixed and changing the observer's location and view direction or keeping the observer fixed and translating, rotating and/or scaling the scene.

Objects are moved or translated in OpenGL by applying a transformation to the current **modelview** matrix. The transformation is a matrix whose particular form depends on whether it is a translation, a rotation or a scaling.

A translation by (x, y, z) is performed by using `glTranslatef(x, y, z)`. This OpenGL call generates the matrix

$$T = \begin{pmatrix} 1 & 0 & 0 & -x \\ 0 & 1 & 0 & -y \\ 0 & 0 & 1 & -z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.10)$$

and applies it to the modelview matrix.

A rotation of θ degrees about the unit-vector axis, (x, y, z) , is performed using `glRotatef( $\theta$ , x, y, z)`. A rotation of θ degrees about the x axis is performed using `glRotate( $\theta$ , 1.0, 0.0, 0.0)`. This OpenGL call generates the matrix

$$R = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.11)$$

and applies it to the modelview matrix.

It is of interest to rotate a scene between vectors u and v about an axis perpendicular to both vectors as illustrated in Fig. 2.9. The rotation axis is formed by generating the cross-product of u and v , namely $u \times v$. The rotation amount θ is found using $\cos(\theta) = u \cdot v / (||u|| \cdot ||v||)$. For the special case that u and v are unit vectors and $u = (0, 0, 1)$, a vector along the z axis, then θ and the rotation axis are given by

$$\theta = \cos^{-1}(v_z) \quad (2.12)$$

$$u \times v = (-v_y, v_x, 0) \quad (2.13)$$

so that the OpenGL call to perform the desired rotation would be `glRotatef( $180 \cos^{-1}(v_z)/\pi, -v_y, v_x, 0$ )`.

A scaling of (a, b, c) meaning that x components are scaled by a , y components are scaled by b , and z components are scaled by c is performed using `glScale3f(a, b, c)`. This OpenGL call generates the matrix

$$R = \begin{pmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.14)$$

and applies it to the modelview matrix. Smokeview uses scaling to view cases with large aspect ratios, tunnel fires for example.

This multiplication is usually performed in hardware by the video card. The modelview matrix is initialized to the identity matrix then multiplied by these translation, rotation or scaling transformation matrices as the scene is moved using the `glTranslate`, `glRotate` and `glScale` OpenGL calls.

2.3.1 Implementation

The desired translation and rotation amounts are communicated between the user and Smokeview using keyboard and/or mouse callback routines. Mouse motion is intercepted by the GLUT mouse callback routines in Smokeview and named `mouse` (first time mouse is clicked) and `motion` (when mouse is pressed and moving). GLUT *informs* the callback of the screen pixel coordinate which Smokeview uses to determine an elevation or azimuth angle or a translation amount depending on which control key (none, CTRL or ALT) is pressed. Smokeview translates and scales the coordinate system defined in an FDS scenario using

$$\hat{x} = (x - x_{\min}) / (xyz_{\max} - xyz_{\min}) \quad (2.15)$$

$$\hat{y} = (y - y_{\min}) / (xyz_{\max} - xyz_{\min}) \quad (2.16)$$

$$\hat{z} = (z - z_{\min}) / (xyz_{\max} - xyz_{\min}) \quad (2.17)$$

where $x_{\min}$ and $x_{\max}$ are the smallest and largest x coordinate considering all meshes. $y_{\min}$, $y_{\max}$, $z_{\min}$ and $z_{\max}$ are defined similarly.

The Smokeview scene is then translated in this coordinate system using `glTranslate` using the mouse position as passed to Smokeview by GLUT.

The Smokeview scene may be rotated about either a vertical or horizontal axis both passing through the scene center. The rotation angle about the horizontal axis (aligned perpendicular to the viewer's line of sight) is called an elevation angle and the rotation angle about the vertical axis is called an azimuth angle. These two angles and the translation amount are used by Smokeview to control the orientation of the scene. Rotations are then implemented in Smokeview using

```
glTranslatef(xcen,ycen,zcen);
glRotatef(YZ_AXIS_angle,cos(az_angle),-sin(az_angle),1.0);
glRotatef(XY_AXIS_angle,0.0,0.0,1.0);
glTranslatef(-xcen,-ycen,-zcen);
```

where `xcen`, `ycen`, `zcen` are the coordinates of the scene center, `YZ_AXIS_angle` is the elevation angle, and `XY_AXIS_angle` is the azimuth angle. The scene center (center of rotation) is translated to the origin, requested rotations are implemented, then the scene is translated back to its original location.

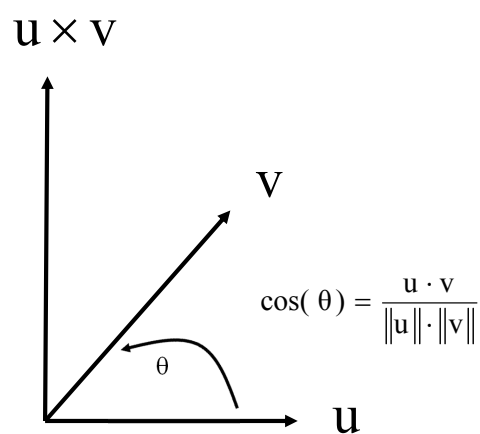


Figure 2.9: Diagram relating the vector $u \times v$ and the angle θ with vectors u, v .

Chapter 3

Visualizing Data

Smokeyview uses color and contours to visualize data quantitatively. Color is used to indicate *how much* of a quantity is at a given location whereas contours are used to identify *where* rather than *how much* a specified quantity occurs.

Coloring methods for visualizing data work by assigning colors based upon data values to various vertices within the scene. These vertices coincide with the FDS grid, either within a horizontal or vertical plane (slice or Plot3D file) or on a blockage exterior (boundary file). The video card then interpolates these colors at the interior of the figure, usually a triangle, formed by the vertices. Coloring variations result from differing methods for interpolating color between the vertices, either interpolating within the 3D color cube or using a 1D texture map (the colorbar).

Smokeyview uses *marching* algorithms for generating contours, marching squares for generating 2D contours and marching cubes [14] for generating 3D contours. *Marching* algorithms work by reducing the general contouring problem to that of finding the contour in an elementary figure such as a square or a cube. Marching squares are used for finding 2D contours in a plane and marching cubes are used for finding isosurfaces in a volume. A contour for a region is then generated by splitting the region into a series of squares for 2D contours or a series of cubes for a 3D contour and finally assembling the contour acquired from all the parts.

Smokeyview uses two methods for representing Smokeyview flow data realistically. The first involves particles, representing smoke flow by tracking tracer particles as they move through the simulation influenced by an underlying velocity field. This velocity field and the particle motion is computed by FDS and communicated to Smokeyview using data files. This was the first method Smokeyview implemented for visualizing smoke flow and was inspired by the software tool named Frames written by James Sims.

The second method for visualizing smoke flow involves a method known as volumetric rendering. Smoke transparency is visualized using the optical properties of soot noting that the amount of light obscured by smoke and hence its shade depends on the quantity of smoke between the observer and the background. Smokeyview then integrates the smoke opacity one grid plane at a time using a simple implementation of Beer's law in the video hardware.

3.1 Coloring Data

Smokeyview uses several methods to visualize data quantitatively involving converting data values to color. The basic procedure is to

1. obtain minimum and maximum data bounds to use in scaling, either through calculation or specification by the user,

2. map data onto integer indices between 0 to 255 ,
3. obtain colors using indices computed in step 2 to index into a color table (a numerical representation of the color bar)
4. display colors using 1D texture maps (or `glColorxx` in the case of particles).

These steps are detailed in the following sections. It is important to point out that the use of 1D texture maps in step 4 enables more detail to be obtained from the visualization due to the way that color interpolations between grid points are performed.

3.1.1 Determining Data Bounds

Smokeview uses three methods for setting data bounds. First, the bounds may be set by the user. This would be useful to ensure consistent coloring when several file types are displayed simultaneously (say slice and boundary files). A second method is to use *percentile* bounds (1st and 99th by default) which are useful when data outliers are present. To find percentile bounds, Smokeview scans the data computing a histogram. It then picks data bounds at a specified percentile levels, 1st and 99th by default. The third method for setting data bounds is to simply pick the global bounds for the data.

3.1.2 Converting data to a color

An index into a color table of size 256 is computed using

$$\text{color index} = \begin{cases} 0 & v < v_{min} \\ 1 + 253(v - v_{min}) / (v_{max} - v_{min}) & v_{min} \leq v \leq v_{max} \\ 255 & v > v_{max} \end{cases} \quad (3.1)$$

where v_{min} and v_{max} are specified data bounds. Each table entry contains 4 components (red, green, blue and opacity). Each component is scaled from 0.0 to 1.0. The data in the colorbar data structure then defines colors (by default) as illustrated by the **bold path** in Fig. 3.1. Other colorbars may be used, for example a color bar containing shades of gray from white to black.

3.1.3 Interpolating Colors

Consider the following code segment for drawing a shaded triangle with red, green and blue vertices:

```
glBegin(GL_TRIANGLE);
glColor3f(1.0,0.0,0.0);
glVertex3f(0.0,0.0,0.0);

glColor3f(0.0,1.0,0.0);
glVertex3f(0.0,1.0,0.0);

glColor3f(0.0,0.0,1.0);
glVertex3f(1.0,0.0,0.0);
glEnd();
```

OpenGL interpolates colors between the vertices and interior to the triangle using the color cube as in Fig. 3.1. For example, if two vertices A and B are colored (0.0, 1.0, 0.5) and (1.0, 0.5, 0.0), then a point half

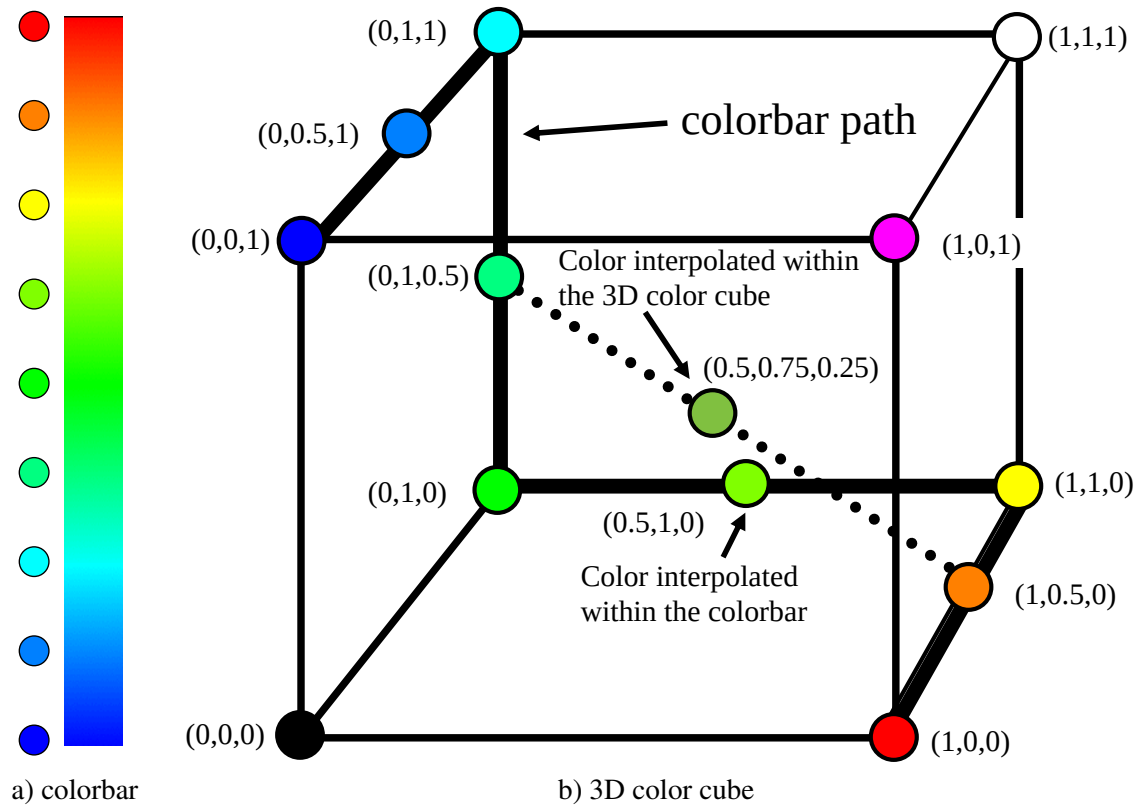


Figure 3.1: The 1D colorbar on the left is mapped onto the 3D color cube along the **bold path** from blue to cyan to green to yellow to red. Colors interpolated within the cube are different than colors interpolated within the colorbar.

way in between would be colored $(0.5, 0.75, 0.25)$, the average of the two colors. This color is the midpoint of the line segment AB interior to the color cube.

Smokeyview version 4 (and earlier) uses this method, interpolating data within the 3D color cube (not the colorbar). As a result, near the fire or wherever there are large temperature gradients, interpolation artifacts occur. For example, if a red $(1,0,0)$ region occurs near a blue $(0,0,1)$ region, the interpolated color halfway in between would be $(0.5, 0.0, 0.5)$, a shade of purple, not in the colorbar. In general, suppose that ci_j is an integer index between 0 and 255 and that $f(ci_j)$ is the ci_j 'th color in the colorbar. Then the interpolated color between the two colors $f(c_1)$ and $f(c_2)$ would be

$$\text{interpolated color} = (f(c_1) + f(c_2))/2 \quad (3.2)$$

This is not a good method for displaying colors related to data since only data on the colorbar have physical meaning. By using 1D texture maps, color indices are interpolated not colors. Therefore, colors displayed in data plots are always contained in the colorbar which is what we want.

Smokeyview interpolates color indices not colors. As a result, interpolated colors are contained in the colorbar. This interpolation method is implemented using 1D texture maps. A 1D texture map is defined using the desired colorbar. A texture coordinate is assigned to each data vertex. Color indices at pixels between vertices are interpolated using the texture coordinate. In general, Smokeyview uses the scheme,

$$\text{interpolated color} = f((c_1 + c_2)/2) \quad (3.3)$$

where c_1 and c_2 are color indices as before.

The following code segment sets up the use of 1D texture map.

```
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_REPLACE);
glEnable(GL_TEXTURE_1D);
glBindTexture(GL_TEXTURE_1D, texture_slice_colorbar_id);
```

The following code segments (simplified) shows an example of drawing a slice in a YZ plane.

```
for(j=jmin; j<jmax; j++){
    for(k=kmin; k<kmax; k++){
        glTexCoord1f( r11); glVertex3f(xplane, yy1, z1);
        glTexCoord1f( r31); glVertex3f(xplane, y3, z1);
        glTexCoord1f(rmid); glVertex3f(xplane,ymid,zmid);

        glTexCoord1f( r31); glVertex3f(xplane, y3, z1);
        glTexCoord1f( r33); glVertex3f(xplane, y3, z3);
        glTexCoord1f(rmid); glVertex3f(xplane,ymid,zmid);

        glTexCoord1f( r33); glVertex3f(xplane, y3, z3);
        glTexCoord1f( r13); glVertex3f(xplane, yy1, z3);
        glTexCoord1f(rmid); glVertex3f(xplane,ymid,zmid);

        glTexCoord1f( r13); glVertex3f(xplane, yy1, z3);
        glTexCoord1f( r11); glVertex3f(xplane, yy1, z1);
        glTexCoord1f(rmid); glVertex3f(xplane,ymid,zmid);
    }
}
glEnd();
```

Figure 3.2 shows the old and new method for coloring. Note that the new method results in crisper, clearer colors. Fig. 3.3 illustrates two methods for interpolating color. Colors are interpolated in Fig. 3.3a

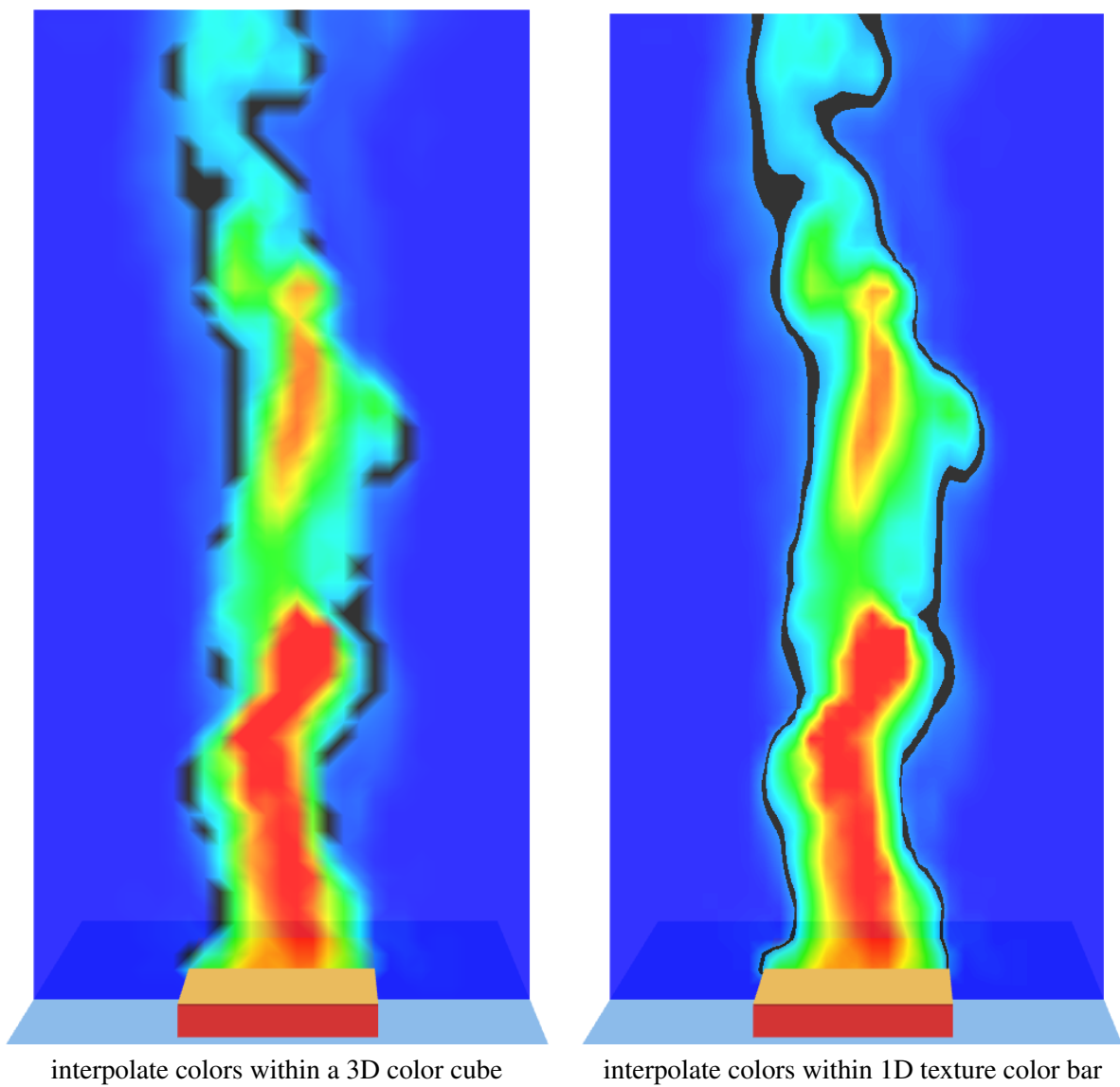


Figure 3.2: Slice file snapshots illustrating old and new method for coloring data.

within the color cube where the colors within the cube have value (r,g,b). Colors are interpolated in Fig. 3.3b with the colorbar.

3.2 2D Contours

The *2D contouring problem* may be stated as: find the region in a 2D plane where a particular value exists (*line contour*) or an interval of values exist (*banded contour*). In each case, the region to be contoured is divided into a series of squares. The problem is solved for each square and assembled to obtain the general solution. The square locations correspond to the grid set up in the FDS input file.

3.2.1 Line contours

Mathematically, the 2D line contouring problem may be expressed as: for some level L , find the line(s) in the desired region where $f(x,y) = L$. Consider this problem for one square and assume that f is known at each square corner. The problem is solved by noting that the value of f at each of the four square corners is either greater than or less than or equal to the contour level L . There are then two states at each of the four corners. As a result, there are 16 cases to consider. These 16 cases may be reduced to 4 after accounting for various symmetries such as rotational, mirror and high/low. High/low symmetry refers to the observation that a case with one corner value above L will look the same as one with 3 corners above L . Figure 3.4 shows these four cases. Corners with blue dots indicate that the solution at this point is greater than L . Red lines indicate the line contouring solution. The algorithm may be summarized as follows:

1. Split the region to be contoured into a number of squares, each square aligned with the underlying FDS grid.
2. For each square:
 - (a) Number the square corners as illustrated in Fig. 3.4.
 - (b) For each cell corner beginning with corner 0: assign 1 if its value exceeds L , 0 otherwise.
 - (c) Use resulting 4 digit binary number ($0 \rightarrow 15$) to determine the case number to be plotted. Using this numbering scheme, the cases are numbered from left to right 0 1, 5 and 3.

3.2.2 Banded contours

The banded contouring algorithm works similarly to the line contouring algorithm discussed previously. The 2D banded contouring problem may be expressed mathematically as given an interval $[L,H]$, find the 2D region where $L \leq f(x,y) \leq H$. The problem is solved by noting that the value of f at each of four square corners is either less than L , greater than H or in between. There are three states at each of four corners resulting in 81 cases to consider. These 81 cases may be reduced to 13 after accounting for various symmetries such as rotational, mirror and high/low. High/low symmetry is defined as before. Figure 3.5 shows these four cases. Each square corner is labeled with an L, M or H denoting that the value of f at that corner is below L , between L and H or above H respectively. The contoured region is bounded by black lines and contains a series of one or more red lines in the interior. The algorithm may be summarized as follows:

1. Split the region to be contoured into a number of squares, each square aligned with the underlying FDS grid.

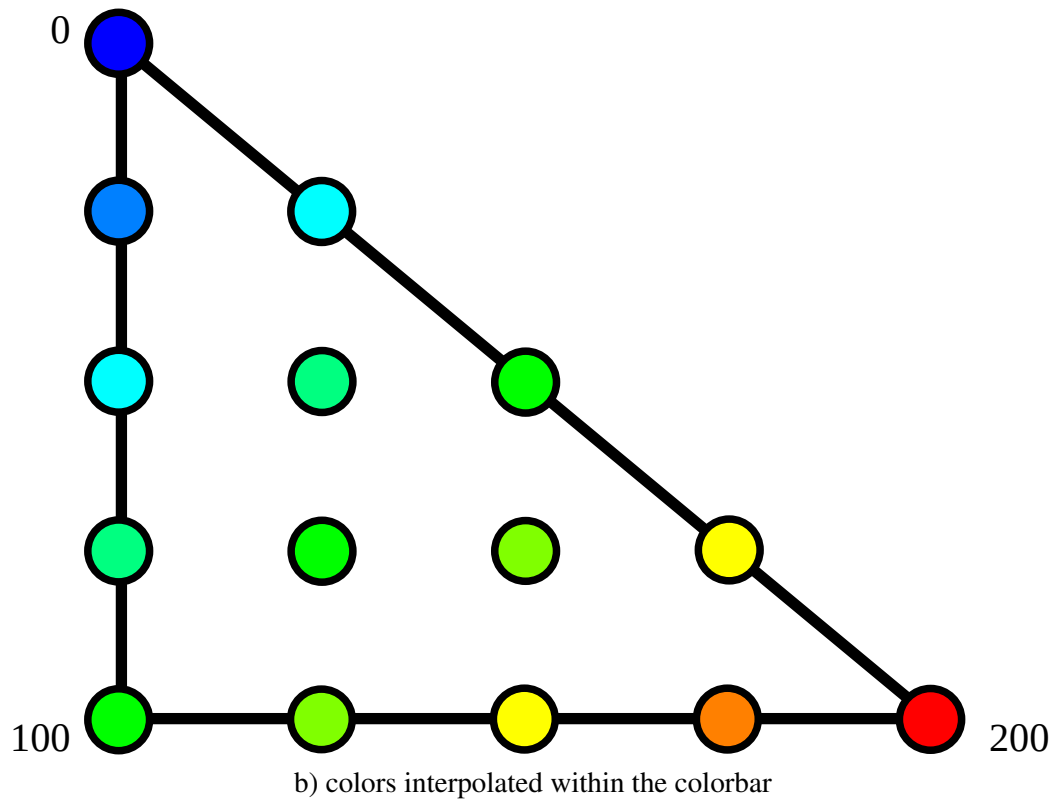
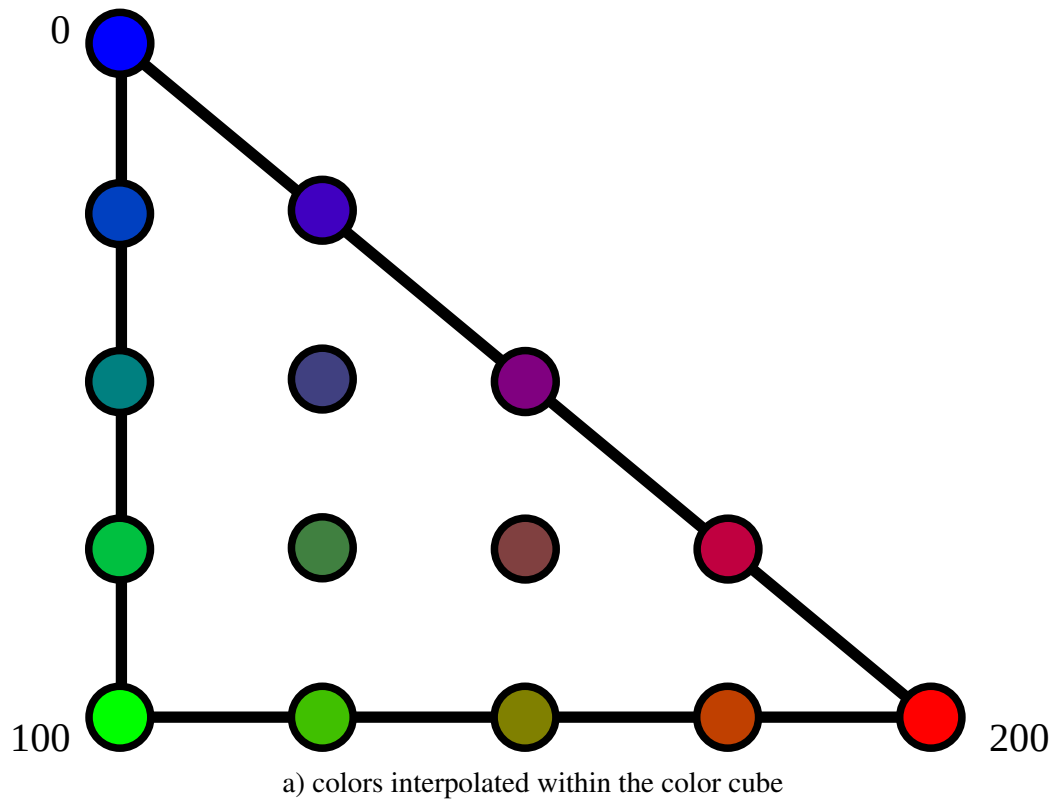


Figure 3.3: Illustration showing colors representing data interpolated two different ways within a triangle: interpolated with the 3D color cube and interpolated with the colorbar

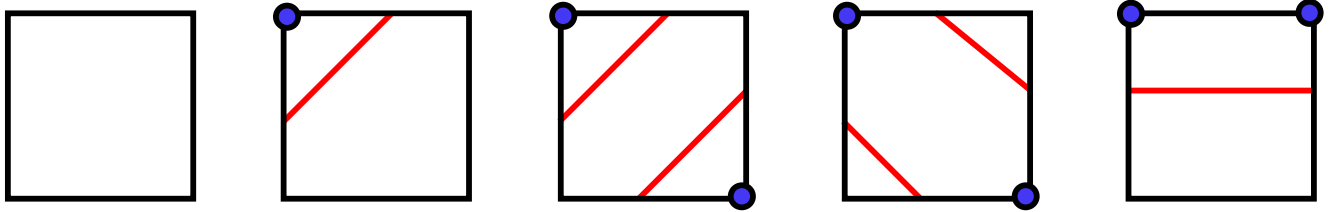


Figure 3.4: 2D line contour canonical forms.

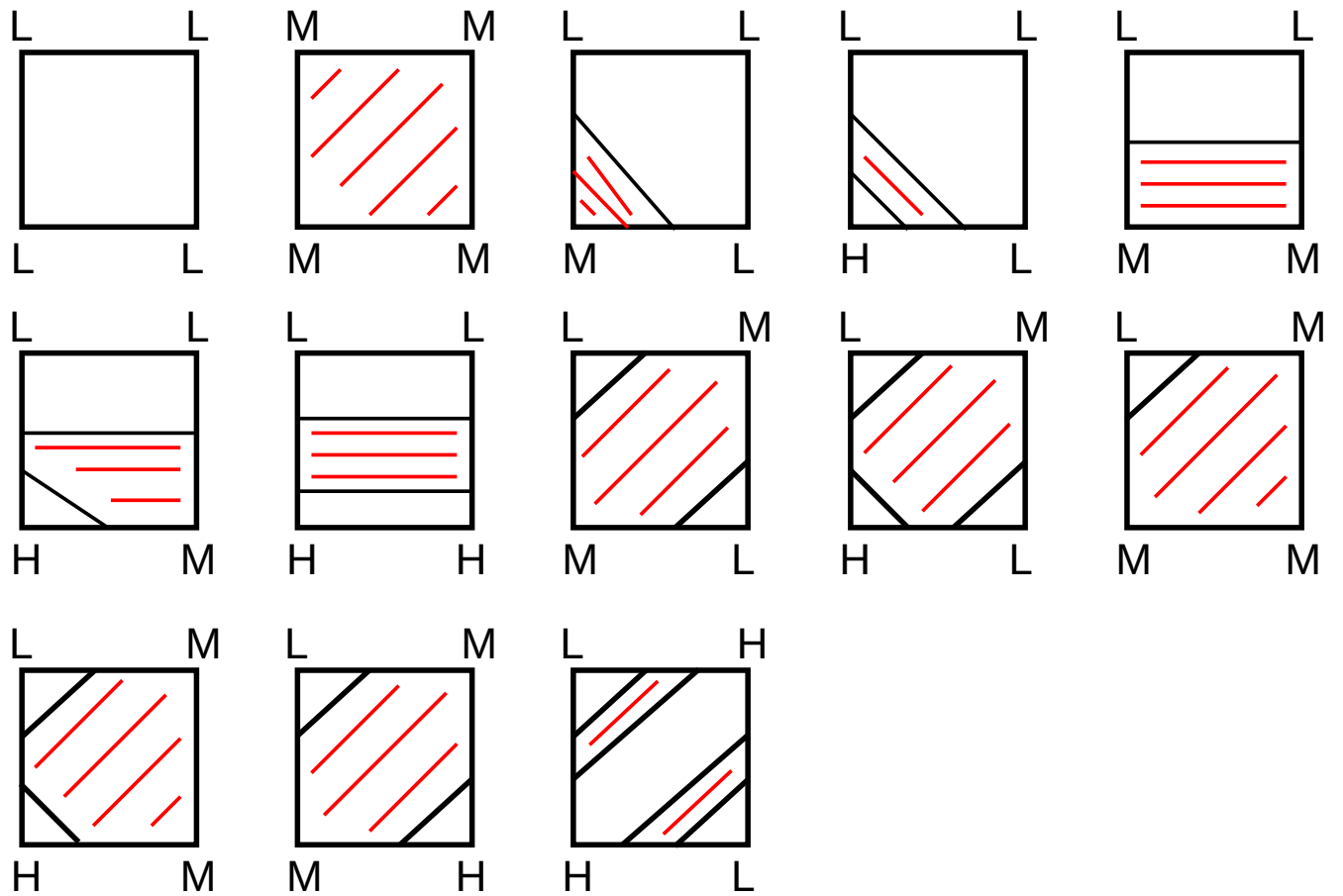


Figure 3.5: 2D band contour canonical forms.

2. For each square:

- (a) Number square corners as illustrated in Fig. 3.5.
- (b) For each cell corner beginning with corner 0: assign 0 if value is less than L , 1 if value is between L and H and 2 if value exceeds H .
- (c) Use resulting 4 digit base 3 number ($0 \rightarrow 80$) to determine the case number to be plotted. The case numbers in the first row are 0, 40, 1, 2 and 3. The case numbers in the second row are 5, 8, 10, 11 and 13. The case numbers in the last row are 14, 16 and 20.

3.3 3D Contours - Isosurfaces

An isosurface is a surface in 3-D space that defines constant values of a dependent variable. For example, if one is interested in investigating regions at a certain temperature, say 100 °C, one would generate an animated isosurface specifying this value.

The isosurfaces are generated at each desired time step using a marching cube algorithm [14] modified to remove ambiguities. A decimation procedure is used to reduce the number of resulting triangles by collapsing nodes of triangles with large aspect ratios and re-triangulating. This makes the isosurface look better and also reduces storage requirements. Figures 3.6 and 3.7 illustrates the use of isosurfaces for visualizing the stoichiometric mixture fraction. These figures show different ways of drawing isosurfaces.

Isosurface uncertainty Data used to generate isosurfaces has uncertainty, therefore the location of the isosurface also has uncertainty. Bounds for this uncertainty may be expressed in terms of data uncertainty and data variation (the smaller the variation the larger the uncertainty). The Smokeview verification guide [3] shows that the uncertainty in isosurface location, Δx , between two node, data value pairs (x_1, T_1) and (x_2, T_2) may be bounded using

$$\left| \frac{\Delta x}{x_2 - x_1} \right| \leq \frac{\max(|\Delta T_1|, |\Delta T_2|)}{|T_2 - T_1|} \quad (3.4)$$

where ΔT_1 and ΔT_2 are the uncertainty in T_1 and T_2 at x_1 and x_2 respectively.

Algorithm for generating an Isosurface The algorithm for generating an isosurface may be summarized as follows:

1. Split region to be contoured into a number of cubes, each cube aligned with the underlying FDS grid.
2. For each cube:
 - (a) Number cube corners from 0 to 7 as illustrated in Fig. 3.8.
 - (b) For each cube corner beginning with corner 0: assign 1 if value exceeds L , 0 otherwise.
 - (c) Use resulting 8 digit binary number ($0 \rightarrow 255$) to determine the case number to be plotted.
 - (d) Triangulate cube surface, storing vertex locations and edge numbers according to case
3. Triangle decimation. Eliminate small triangles (any triangle with two or more vertices *closest* to the same node). Replace removed triangle vertices with a vertex with coordinates that are the average of the three removed vertices coordinates. Re-triangulate using this new vertex. This process is illustrated in Fig. 3.9.
4. Estimate normal directions (to be associated with a vertex). For each vertex:
 - (a) determine triangles sharing the vertex
 - (b) determine the normal vector (normalized with length 1.0) of each triangle using the vertex
 - (c) construct a harmonic weighted average of triangle normal vectors where all weights sum to 1.0 and each weight is inversely proportional to the corresponding triangle area
 - (d) store the resulting average along with vertex data

Vertex normals in 3D An averaged normal vector, V_i , for vertex i may then be determined by summing over all triangles n connected to a vertex i as in

$$V_i = \sum_n \frac{U_n}{A_n} / \sum_n \frac{1}{A_n} = \sum_n \frac{u_n \times v_n}{\|u_n \times v_n\|^2} / \sum_n \frac{1}{\|u_n \times v_n\|} \quad (3.5)$$

where for the n 'th triangle, u_n and v_n are two sides, $A_n = \|u_n \times v_n\|$ is the triangle area and $U_n = u_n \times v_n / A_n$ is a unit vector normal to the triangle.

Vertex normals in 2D The use of inverse area weights or harmonic averages to construct average normal vectors at isosurface vertices is justified by the following two dimensional example. As illustrated in Fig. 3.10, consider the quadratic $y(x) = A + Bx + Cx^2$ with $y(-\Delta x_2) = 0$, $y(0) = 1$ and $y(\Delta x_1) = 0$. The slope at $x = 0$ is given by $y'(0) = B$. Likewise, the slope of the vector perpendicular to this curve at $x = 0$ is $-1/B$. The coefficient B may be determined from the two simultaneous equations $y(\Delta x_1) = 0$ and $y(-\Delta x_2) = 0$ (note that $A = 1$ since $y(0) = 1$) or

$$1 + B\Delta x_1 + C\Delta x_1^2 = 0 \quad (3.6)$$

$$1 - B\Delta x_2 + C\Delta x_2^2 = 0 \quad (3.7)$$

which has solution

$$B = \frac{\Delta x_1^2 - \Delta x_2^2}{\Delta x_1 \Delta x_2 (\Delta x_1 + \Delta x_2)} = \frac{\Delta x_1 - \Delta x_2}{\Delta x_1 \Delta x_2} \quad (3.8)$$

The slope N of the normal at $x = 0$ is then given by

$$N = -\frac{1}{B} = \frac{1}{1/\Delta x_1 - 1/\Delta x_2} \quad (3.9)$$

The slope N_1 of the normal to the line segment between $(0, 1)$ and $(\Delta x_1, 0)$ is $N_1 = \Delta x_1$. The slope N_2 of the normal to the line segment between $(-\Delta x_2, 0)$ and $(0, 1)$ is $N_2 = -\Delta x_2$. Therefore the average slope N may be expressed in terms of N_1 and N_2 as

$$N = \frac{1}{1/N_1 + 1/N_2} \quad (3.10)$$

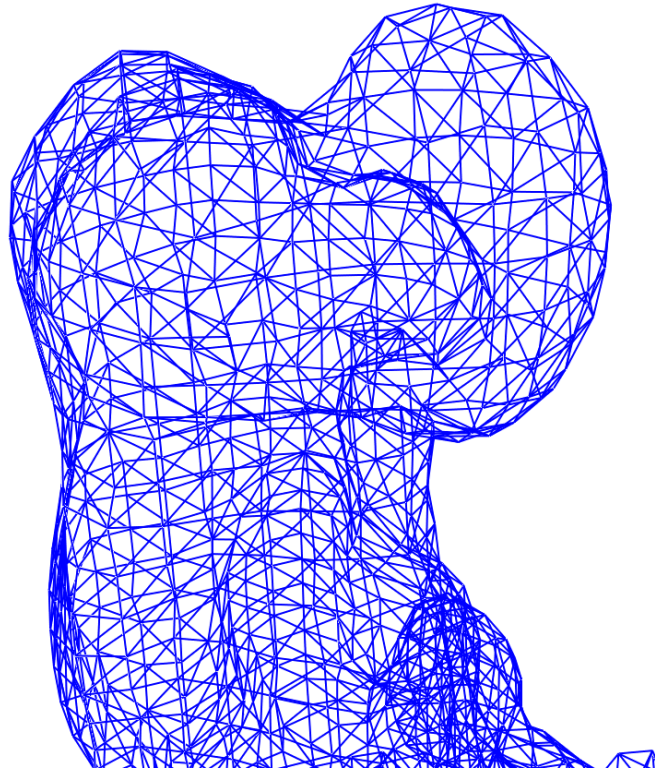
Isosurface opacity Consider a portion of an isosurface oriented at an angle θ from the observer, as illustrated in Figure 3.11. The vector $\vec{n}$ represents the direction perpendicular from the surface and the vector $\vec{v}$ represents the direction from the surface to the observer. The distance $\Delta \hat{x}$ through the surface from the point of view of the observer is then given by

$$\frac{\Delta \hat{x}}{\Delta x} = \frac{1}{\cos \theta} = \frac{\|\vec{n}\| \|\vec{v}\|}{\vec{n} \cdot \vec{v}} \quad (3.11)$$

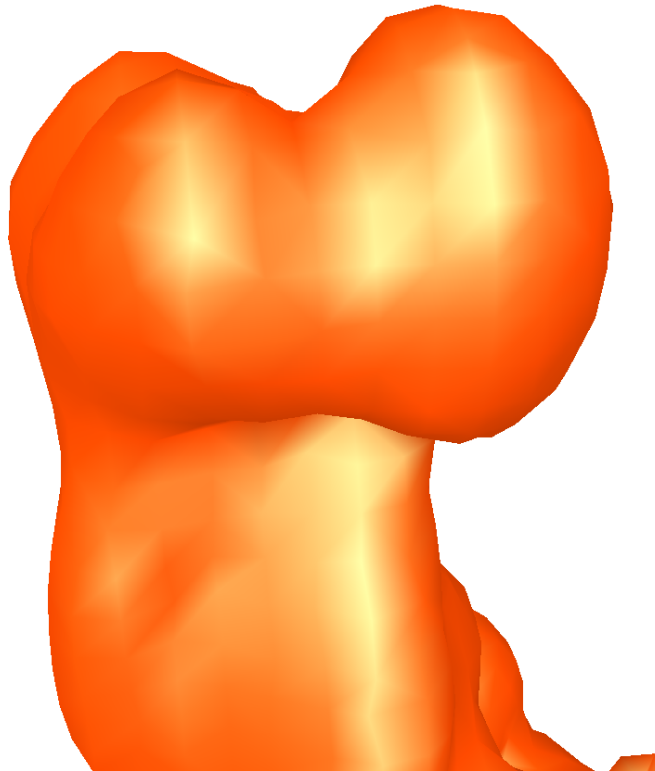
where θ is the angle between $\vec{n}$ and $\vec{v}$. Let $\alpha = 1 - \exp(-A\Delta x)$ represent the opacity corresponding to a surface thickness Δx . Then the opacity when the surface is oriented an angle θ with respect to the observer is given



Figure 3.6: Snapshot of an isosurface of temperature at 100 °C (212 °F).



outline view



solid view

Figure 3.7: Snapshot of an isosurface of temperature at 100 °C (212 °F).

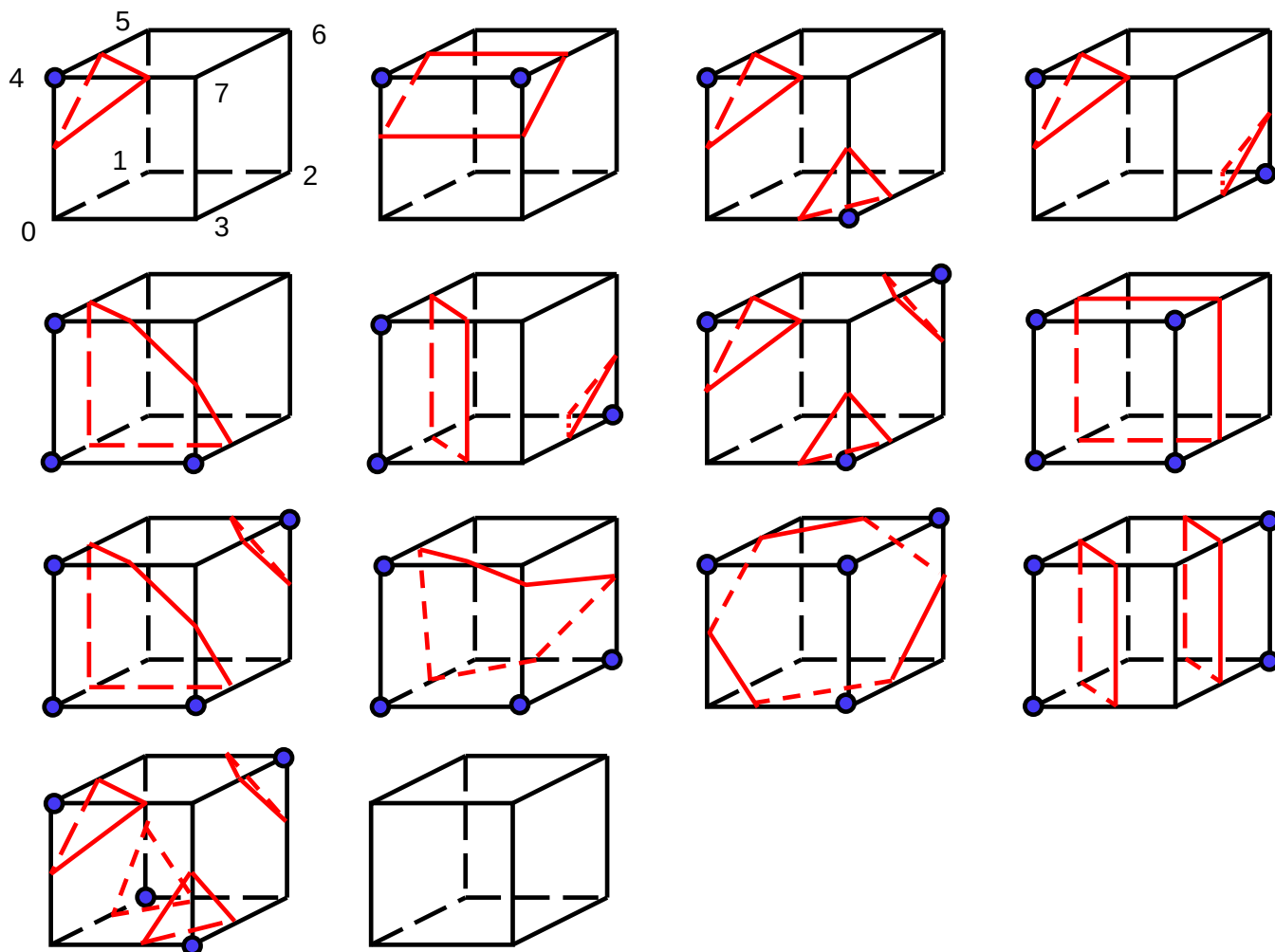
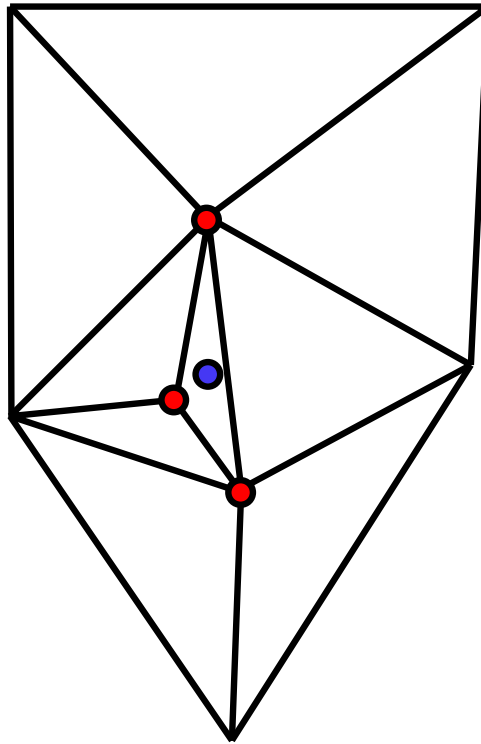
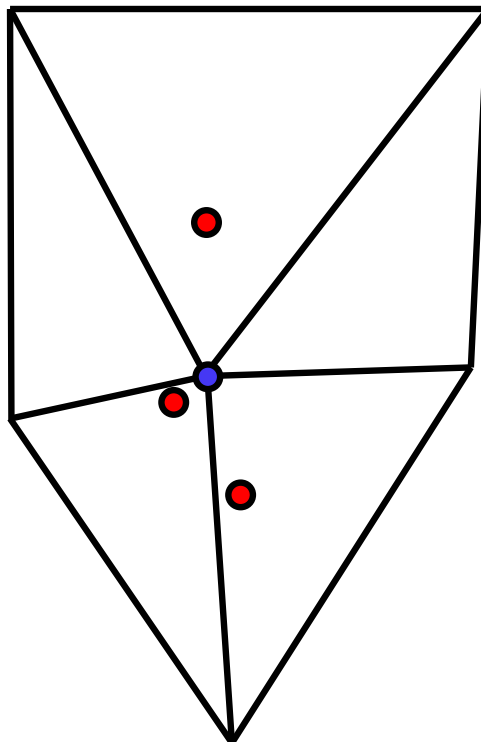


Figure 3.8: 3D isosurface canonical forms. Dots occur at corners where the data value is greater than the isosurface value. Other corners are below the isosurface value. Red polygons intersect cube edges at the isosurface value. The red polygon (isosurface) NEVER intersects an edge with two or zero dots on the ends.



before decimation



after decimation

Figure 3.9: Example of triangle decimation. Triangle with red dots is removed. Region is re-triangulated by replacing any edges connected to a red dot with the blue dot (average position of removed red dot).

by

$$\begin{aligned}
\hat{\alpha} &= 1 - e^{A\Delta\hat{x}} \\
&= 1 - (e^{A\Delta x})^{\frac{\Delta\hat{x}}{\Delta x}} \\
&= 1 - (1 - \alpha)^{\frac{\Delta\hat{x}}{\Delta x}} \\
&= 1 - (1 - \alpha)^{\frac{\|\vec{n}\| \cdot \|\vec{v}\|}{\vec{n} \cdot \vec{v}}}
\end{aligned}$$

Figure 3.12 shows an example of an isosurface using this equation for modeling opacity.

3.4 Particle Systems

3.4.1 Massless Particles

FDS uses particles as tracer elements to allow one to visualize flow. FDS also uses particles as droplets to model fire suppression or as fuel elements that can be transported and burned, for example, as embers emitted from a burning tree. Particle positions are determined in FDS using the differential equation

$$\frac{dx_p}{dt} = V(x_p, t) \quad (3.12)$$

where x_p represents particle position and V represents the velocity field.

The assumption this model uses to compute particle flow is that the drag force on the particle is large compared to mg , the force due to gravity.

Figure 3.13 shows particles represented as particles and as streaks. A particle streak visualizes where a particle is located over a period of time. The streak length is specified as time. Static particle images are not effective at displaying motion. For example, the particle image in Fig. 3.13a does not show particle motion. Streak lines, however, in Fig. 3.13b shows curved motion due to interior obstructions. Particle size is specified using

```
glPointSize(partpointsize);
```

The floating point value of `partpointsize` is specified using the File/Bounds dialog box. Streak line length is specified in terms of time, a certain number of seconds before the current display time. Streak length is also specified in the File/Bounds dialog box.

3.5 Computing and Visualizing Fractional Effective Dose data

The fractional effective dose (FED), developed by Purser [15], is a measure of human incapacitation due to combustion gases. FED index data is computed by Smokeview using CO, CO₂ and O₂ gas concentration data computed by FDS. Future work involves incorporating other constituents such as soot or HCN. Smokeview obtains this data from FDS using slice files.

The total FED is computed here in terms of FED components for CO and O₂. A hyper-ventilating factor due to CO₂ is applied to the FED for CO. The total FED is then given by

$$FED_{tot} = FED_{CO} \times HV_{CO_2} + FED_{O_2} \quad (3.13)$$

Other terms involving CN, NO_x and irritants are neglected. The fraction of an incapacitating dose due to CO, FED_{CO} is calculated using

$$\text{FED}_{\text{CO}}(t) = \int_0^t 2.764 \times 10^{-5} (C_{\text{CO}}(t))^{1.036} dt \quad (3.14)$$

where t is time in minutes and C_{CO} is the CO concentration (ppm). The fraction of an incapacitating dose due to low O₂ hypoxia, FED_{O₂}, is calculated using

$$\text{FED}_{\text{O}_2}(t) = \int_0^t \frac{dt}{\exp[8.13 - 0.54(20.9 - C_{\text{O}_2}(t))]} \quad (3.15)$$

where t is time in minutes and C_{O_2} is the O₂ concentration (volume per cent). The hyperventilation factor induced by carbon dioxide, HV_{CO₂}, is calculated using

$$\text{HV}_{\text{CO}_2}(t) = \frac{\exp(0.1903 C_{\text{CO}_2}(t) + 2.0004)}{7.1} \quad (3.16)$$

where t is time in minutes and C_{CO_2} is the CO₂ concentration (percent).

3.5.1 FED example

Assuming that $C_{\text{CO}}(t) = \text{CO}$, $C_{\text{CO}_2}(t) = \text{CO}_2$ and $C_{\text{O}_2}(t) = \text{O}_2$ are constant, equations 3.14, 3.15 and 3.16 reduce to

$$\text{FED}_{\text{CO}}(t) = 2.764 \times 10^{-5} \text{CO}^{1.036} t \quad (3.17)$$

$$\text{FED}_{\text{O}_2}(t) = \exp(-8.13 + 0.54(20.9 - \text{O}_2))t \quad (3.18)$$

$$\text{HV}_{\text{CO}_2}(t) = \frac{\exp(0.1903 \text{CO}_2 + 2.0004)}{7.1} \quad (3.19)$$

so that the total FED is given by

$$\text{FED}_{\text{tot}} = \text{FED}_{\text{CO}}(t) \times \text{HV}_{\text{CO}_2}(t) + \text{FED}_{\text{O}_2}(t) \quad (3.20)$$

$$= \left(2.764 \times 10^{-5} \text{CO}^{1.036} \times \frac{\exp(0.1903 \text{CO}_2 + 2.0004)}{7.1} + \exp(-8.13 + 0.54(20.9 - \text{O}_2)) \right) t \quad (3.21)$$

Figure 3.14 presents two FED computations where CO, CO₂ and O₂ are constant. The first for a smoke filled room with CO=10 000 ppm, CO₂ = 5% and O₂ = 10% and the second for a room with ambient conditions with CO=0 ppm, CO₂ = 0.04% and O₂ = 21%

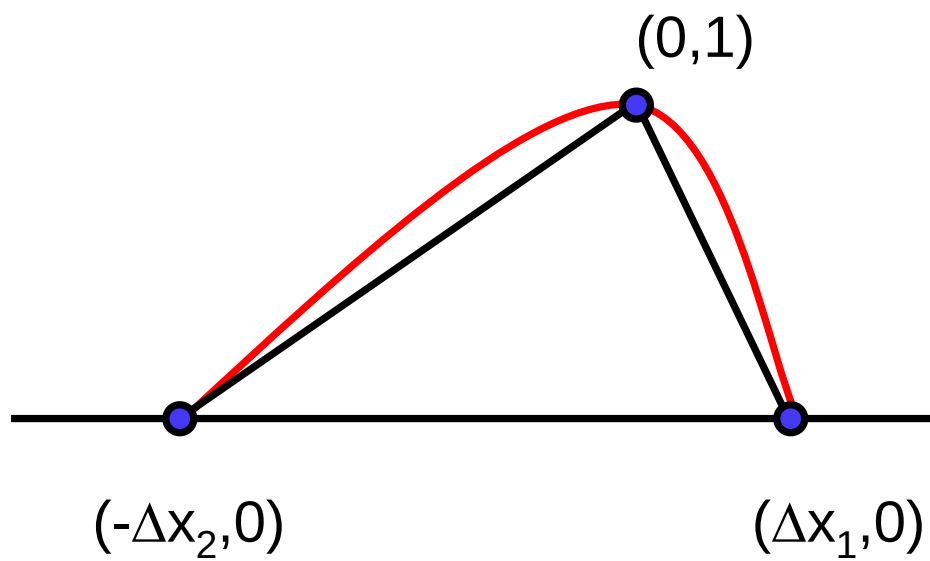


Figure 3.10: Setup for determining the slope of a smooth curve passing through three points.

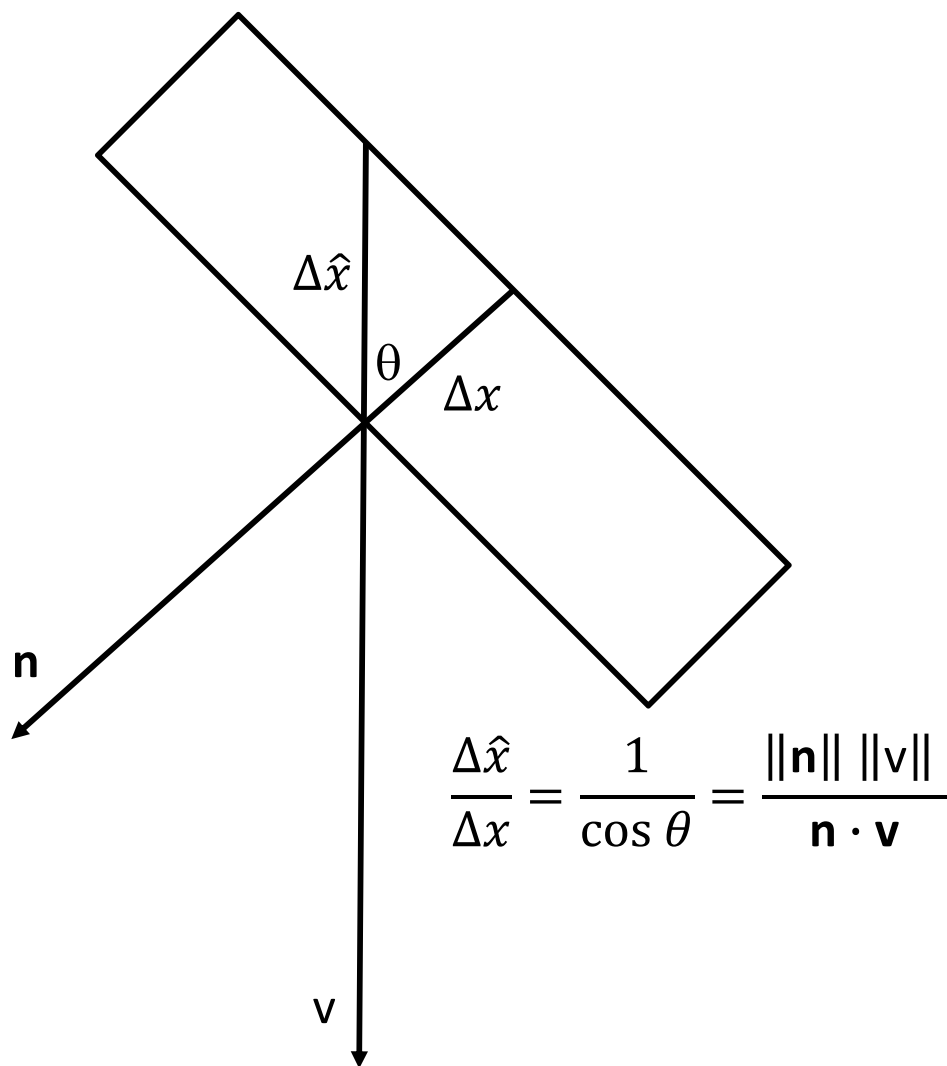


Figure 3.11: Setup for determining isosurface opacity as a function of orientation. The vectors $\vec{n}$ and $\vec{v}$ represent the direction perpendicular to the surface and the direction from the surface to the observer.

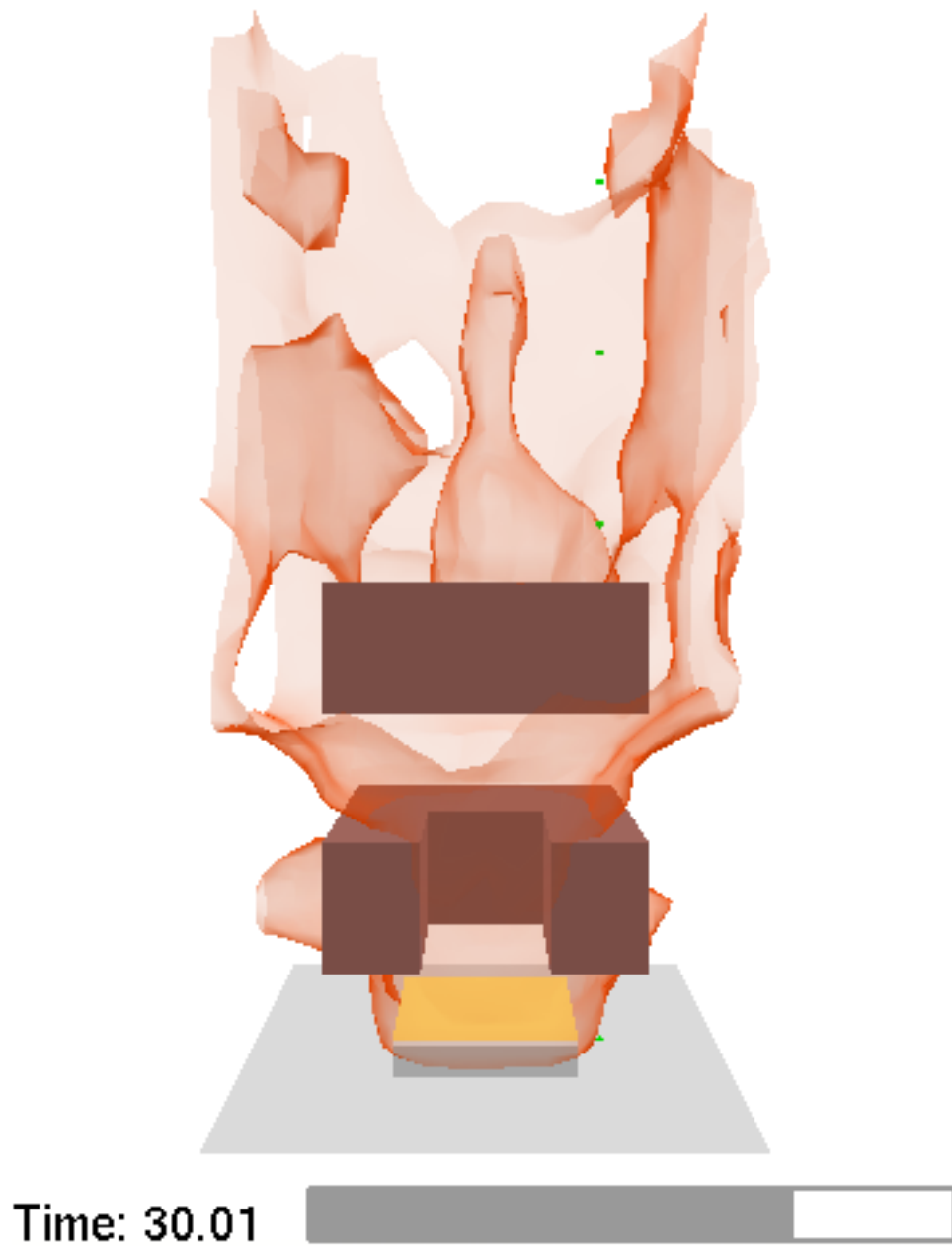


Figure 3.12: Isosurface with variable opacity. The isosurface opacity changes as a function of orientation with respect to the observer.

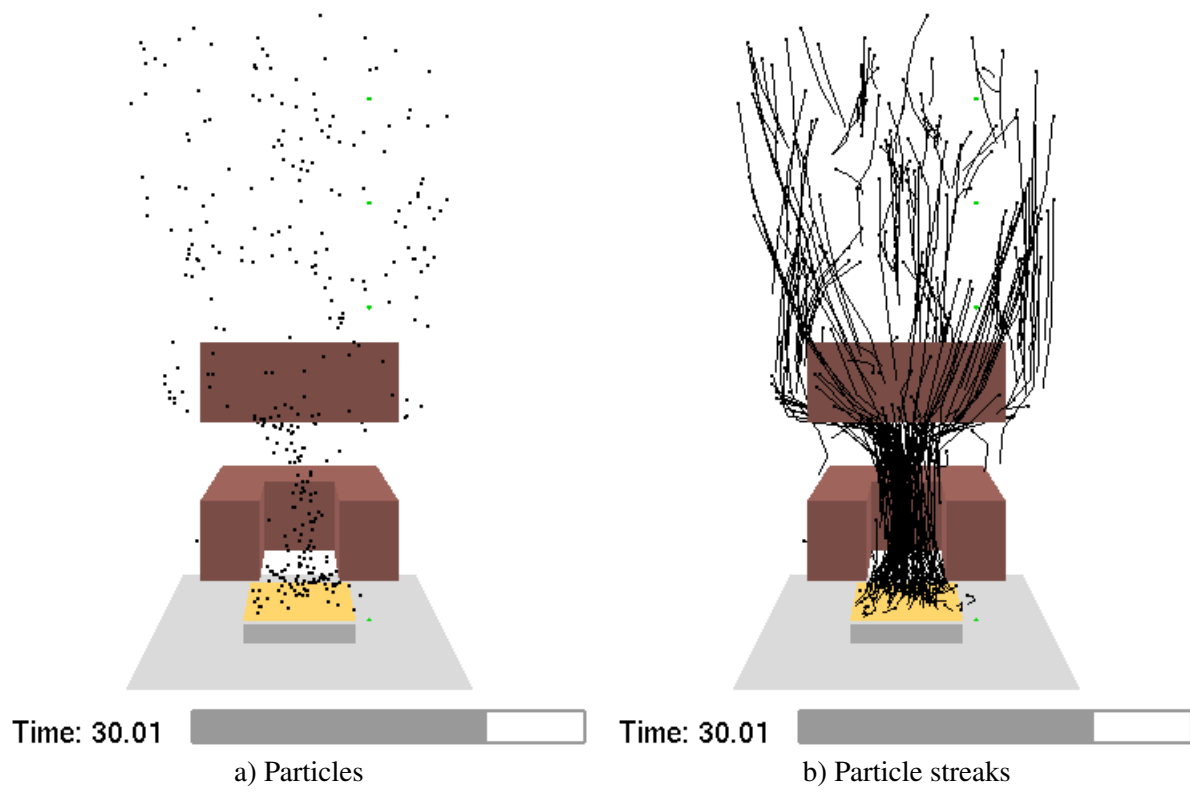
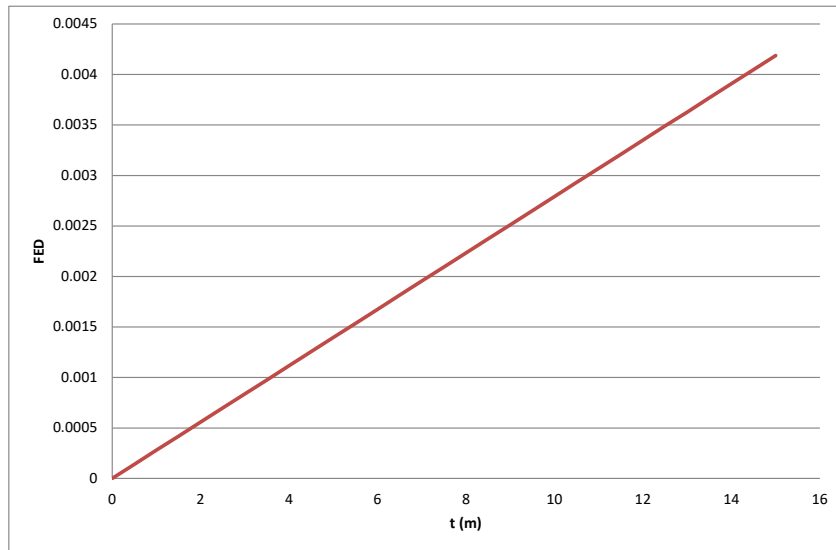
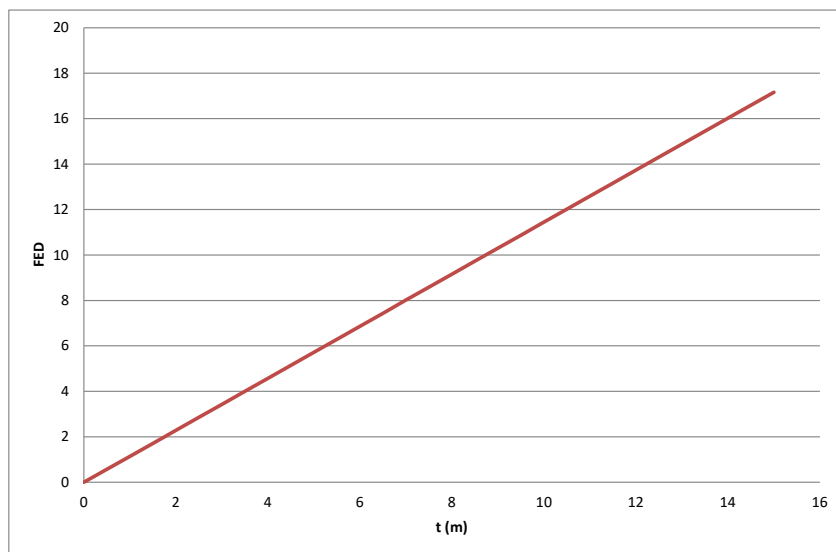


Figure 3.13: Plume flow visualized using particles and particle streaks.



a) ambient



b) smoke filled

Figure 3.14: Plot of FED vs. time for ambient and smoke filled rooms.

Chapter 4

Visualizing Smoke and Fire

4.1 Introduction

This chapter documents the physics and associated numerical algorithms used by Smokeview [1] to visualize smoke and fire. Smoke color and opacity are visualized using quantitative physics based methods. Flame color at present is visualized using an arbitrary user specified color palette where color is mapped to gas temperature. Future work involves mapping color to a blackbody temperature curve enabling a more quantitative choice of color for this application.

Realistic visualization of fire calculation methods are important for applications where one wishes to observe qualitative effects of fire and smoke rather than determine quantitative characteristics of data such as temperature or velocity. This would be the case for a fire fighter using a computer based fire fighting simulator. Realistic visualization methods, however, complement but do not replace other more traditional visualization methods such as 2D contouring or 3D iso-surfacing which are better suited for quantitatively analyzing data.

Complete methods for visualizing smoke and fire data taking into account interactions between light and smoke require the solution of the radiation transport equation (RTE) [16] also called the volume rendering equation in visualization literature. [17] This equation models how light is affected after interacting with smoke, a participating medium. In particular, Smokeview uses the RTE to account for extinction (absorption plus out-scattering) by the smoke and emission from the fire.

The form of the RTE used by Smokeview to model smoke and fire appearance is identical to that used by FDS to model radiative heat transfer. Smokeview uses an extinction coefficient appropriate for visible light while FDS use one appropriate for infrared wavelengths of light. With the proper extinction coefficient, however, Smokeview can also view smoke at other wavelengths, simulating a thermal imager, for example. Smokeview solves the RTE assuming a gray gas environment. This is the default solution method for FDS. One other important difference is that Smokeview requires a solution at only one point at a time (any arbitrary point though), the observer's viewpoint, while FDS requires a radiation field, a solution at all points within the solution domain. Even so, the solution of this integro-differential equation for visualization requires significant computation and memory resources. Approximations are required in order to display smoke and fire at interactive frame rates. The primary approximation is to take advantage of the low albedo character of smoke allowing one to either simplify or eliminate scattering terms in the RTE.

Two techniques discussed for visualizing smoke are slice rendered and volume rendered methods [17, 18]. These methods both solve a form of the RTE equation. The integrated quantity in both cases is radiance, the intensity of light seen by the observer. They differ in how the integration path is partitioned.

The first approach, a slice rendering method, splits the integration path at grid planes within a 3D mesh. There is one partially transparent slice for each plane of simulated data. Planes are drawn through the data

along the coordinate planes or diagonally to these planes. The particular plane drawn is chosen to be the one most perpendicular to the viewer's line of sight. The resulting partially transparent slice planes are drawn individually and combined by the video hardware to form one image.

The separation distance between slice planes becomes smaller as more planes are used to simulate a case. As a result, the computed opacity values are subject to increased round off error due to finite precision arithmetic. In fact, if these planes are sufficiently close, the computed opacities truncate to zero. In this situation, volume rendering methods are required.

The second approach, a volume rendering method, also integrates a simplified form of the radiation transport equation, but over the entire data mesh. The integration occurs from the front of the solution domain (relative to the observer) to the back, rather than across just one slice plane. By performing the entire integral at once instead of in pieces, the volume rendered method does not suffer from the round off errors of the slice rendered method. The intermediate computational terms in the volume rendered method are stored using full precision arithmetic rather than 8 bits. A volume rendered method then computes opacity across multiple grid planes. As a result, there is only one plane of data displayed. The video hardware is again exploited, but this time to compute a line integral for each pixel in the observer's view. Opacities and color for both methods are computed using transfer functions using soot density and temperature data obtained from a fire simulation.

This chapter is organized as follows. A model for visualizing smoke and fire is discussed. This model, the radiative transport equation (RTE), is simplified in several ways, one of which gives the Beer-Lambert law. Two methods are then discussed for visualizing smoke both using a form of the RTE. The first method, slice rendering, uses a series of partially transparent slices to represent smoke and fire. The second method, volume rendering, solves the RTE over the entire domain. Finally, several areas for which the visualization methods may be improved are given.

4.2 Radiation Transport Equation

The model used here to visualize smoke is the radiation transport equation (RTE) [16]. This equation uses radiance to represent smoke appearance. Radiance has units of Watts per square meter per unit solid angle $W/(sr \cdot m^2)$. The solid angle accounts for the fact that a light source appears brighter if it emits a given amount of light through a smaller cross-sectional area. This has the interesting implication, ignoring atmospheric effects and as pointed out in Ref. [19], that the sun's radiance appears the same observed from Earth as from Mars. The diminished heat flux (W/m^2 on Mars due to the increased distance from the sun is exactly offset by the reduced solid angle (sr) that the sun's disk subtends. From a visualization perspective, this implies that image radiance does not depend on distance from the observer unless a participating medium is present to absorb or scatter light. The radiation transport equation discussed in this section models the change in radiance due to these factors.

The radiation transport equation is used to calculate radiance due to one or more light sources within a region possibly containing a participating medium such as smoke [16]. The change in radiance along a ray with direction ω at any one instant and wavelength may be expressed using

$$(\omega \cdot \nabla) C(x, \omega) = - \underbrace{\sigma_a(x) C(x, \omega)}_{\text{absorption}} - \underbrace{\sigma_s(x) C(x, \omega)}_{\text{out-scattering}} + \underbrace{\sigma_a(x) C_e(x, \omega)}_{\text{emission}} + \underbrace{\sigma_s(x) \int_{4\pi} p(x, \omega, \omega') C_i(x, \omega') d\omega'}_{\text{in-scattering}} \quad (4.1)$$

where $C(x, \omega)$ represents the radiance at x along a direction ω . As illustrated in Fig. 4.1, the right hand side of (4.1) is split into four components accounting for absorption, in and out scattering and emission where $\sigma_a(x)$ is the absorption coefficient, $\sigma_s(x)$ is the scattering coefficient, $C_e(x, \omega)$ is the radiance emitted

at x along a direction ω and $p(x, \omega, \omega')$ is the fraction of light moving along direction ω' scattered along direction ω . Absorption and out-scattering cause radiance to decrease while emission and in-scattering cause radiance to increase. The radiance terms C , C_e and C_i have units of $\text{W}/(\text{m}^2 \cdot \text{sr})$. The coefficients σ_a and σ_s have units of $1/\text{m}$ and specify the time and location dependent change per unit length to the radiance term to which they are applied.

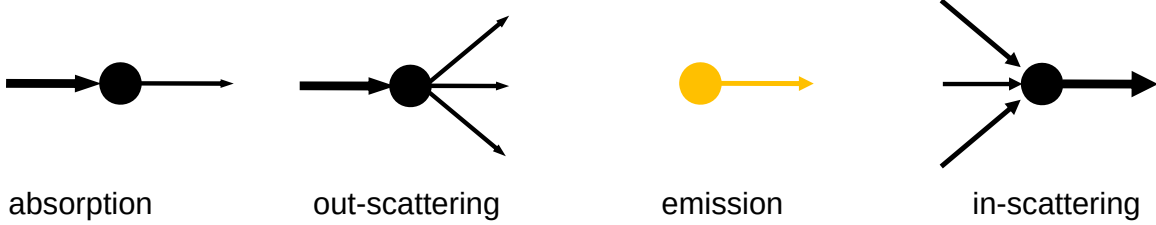


Figure 4.1: Diagram illustrating components of the radiation transport equation. Absorption and out-scattering terms decrease radiance. Emission and in-scattering terms increase radiance.

4.2.1 Approximating the Radiation Transport Equation

The RTE may be simplified in several ways depending on which terms are included or ignored. This section derives an approximate solution in which the in-scattering integral term in (4.1) is neglected and the absorption and out-scattering coefficients are combined (both are loss terms) using $\sigma_t(x) = \sigma_a(x) + \sigma_s(x)$. This simplification then only includes interactions between light and smoke due to absorption, out-scattering and emission. Note that the Beer-Lambert law results if the emission term is also dropped.

Equation (4.1) is then approximated by neglecting the integral term and using $\sigma_t(x) = \sigma_a(x) + \sigma_s(x)$ to obtain

$$\frac{dC}{dx}(x) = -\sigma_t(x)C(x) + \sigma_a(x)C_e(x) \quad (4.2)$$

$$C(x_0) = C_0 \quad (4.3)$$

This equation may be solved by rearranging terms and applying the integrating factor $\exp(\int_{x_0}^x \sigma_t(s) ds)$ obtaining

$$\exp\left(\int_{x_0}^x \sigma_t(s) ds\right) \left(\frac{dC}{dx}(x) + \sigma_t(x)C(x)\right) = \exp\left(\int_{x_0}^x \sigma_t(s) ds\right) \sigma_a(x)C_e(x) \quad (4.4)$$

$$\frac{d}{dx} \left(\exp\left(\int_{x_0}^x \sigma_t(s) ds\right) C(x) \right) = \exp\left(\int_{x_0}^x \sigma_t(s) ds\right) \sigma_a(x)C_e(x) \quad (4.5)$$

Integrating both sides and substituting the integration limits results in

$$\exp\left(\int_{x_0}^x \sigma_t(s) ds\right) C(x) \Big|_{x_0}^{x_N} = \int_{x_0}^{x_N} \exp\left(\int_{x_0}^x \sigma_t(s) ds\right) \sigma_a(x)C_e(x) dx \quad (4.6)$$

$$\exp\left(\int_{x_0}^{x_N} \sigma_t(s) ds\right) C(x_N) - C_0 = \int_{x_0}^{x_N} \exp\left(\int_{x_0}^x \sigma_t(s) ds\right) \sigma_a(x)C_e(x) dx \quad (4.7)$$

Solving for $C(x_N)$ after noting that $\int_{x_0}^x \sigma_t(s) ds - \int_{x_0}^{x_N} \sigma_t(s) ds = -\int_x^{x_N} \sigma_t(s) ds$ results in

$$C(x_N) = \exp\left(-\int_{x_0}^{x_N} \sigma_t(s) ds\right) C_0 + \int_{x_0}^{x_N} \exp\left(-\int_x^{x_N} \sigma_t(s) ds\right) \sigma_a(x) C_e(x) dx \quad (4.8)$$

which may be simplified to

$$C(x_N) = \tau(x_0, x_N) C_0 + \int_{x_0}^{x_N} \tau(x, x_N) \sigma_a(x) C_e(x) dx \quad (4.9)$$

after defining $\tau(a, b)$ as

$$\tau(a, b) = \exp\left(-\int_a^b \sigma_t(s) ds\right) \quad (4.10)$$

which represents the optical depth between a and b . As noted earlier, if the emission term is neglected and $\sigma_t(x) = \sigma_t$ is constant over a path with length $L = x_N - x_0$, then (4.9) simplifies to

$$\frac{C(x_N)}{C_0} = \exp(-\sigma_t L) \quad (4.11)$$

which is the Beer-Lambert law.

4.2.2 Discretizing the Radiation Transport Equation

The approximate RTE solution given in (4.9) is discretized by converting integral terms into Riemann sums. Figure 4.2 illustrates the terms used to perform these discretizations. The path is split into N parts each with length $\Delta x = (x_N - x_0)/N$. The coordinate system is set up so that the initial radiance, C_0 , is located at x_0 , most distant from the observer and the final radiance, C_N , is located at x_N closest to the observer.

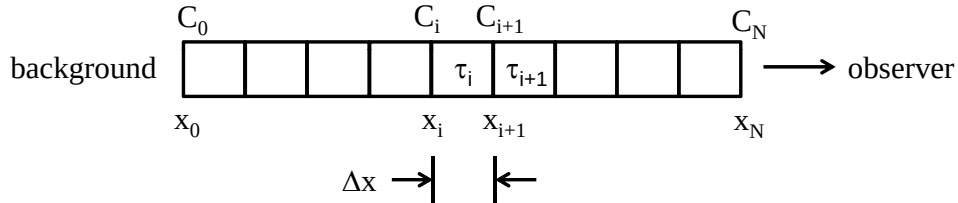


Figure 4.2: Setup for discretizing the equations used to model radiance within a column of 3D smoke data. The transparency across the interval from x_i to x_{i+1} is τ_i . The transparency across the intervals from x_i to the observer is the product of individual transparencies or $\tau_i \tau_{i+1} \cdots \tau_{N-1}$

The optical depth, $\tau(a, b)$, defined in (4.10) is discretized using a Riemann sum after defining sample points $s_j = x_0 + j\Delta s$ for $j = 0$ to N with spacing $\Delta s = (x_N - x_0)/N$ to obtain

$$\tau_i^{N-1} = \tau(x_i, x_N) = \exp\left(-\int_{x_i}^{x_N} \sigma_t(s) ds\right) \approx \exp\left(-\sum_{j=i}^{N-1} \sigma_t(s_j) \Delta s\right) \quad (4.12)$$

$$= \prod_{j=i}^{N-1} \exp(-\sigma_t(s_j) \Delta s) = \prod_{j=i}^{N-1} \tau_j \quad (4.13)$$

where $\tau_j = \exp(-\sigma_j(s_j)\Delta s)$ represents the transparency over one discretization interval. For $i = N - 1$ to 0, the optical depth τ_i^{N-1} may be computed recursively using

$$\tau_i^{N-1} = \tau_{i+1}^{N-1} \tau_i \quad (4.14)$$

where the recursion is initiated with $\tau_N^{N-1} = 1$. Substituting $1 - \alpha_i^{N-1} = \tau_i^{N-1}$ and $1 - \alpha_i = \tau_i$ into (4.14) gives

$$1 - \alpha_i^{N-1} = (1 - \alpha_{i+1}^{N-1})(1 - \alpha_i) = 1 - \alpha_{i+1}^{N-1} - \alpha_i + \alpha_{i+1}^{N-1} \alpha_i \quad (4.15)$$

which simplifies to

$$\alpha_i^{N-1} = \alpha_{i+1}^{N-1} + (1 - \alpha_{i+1}^{N-1}) \alpha_i \quad (4.16)$$

Similarly, the radiance given by the RTE solution $C(x_N)$ in (4.9) may be discretized to obtain

$$C_N = \tau_0^{N-1} C_0 + \sum_{i=0}^{N-1} \tau_i^{N-1} \sigma_{a,i} C_{e,i} \Delta x \quad (4.17)$$

where $\sigma_{a,i} = \sigma_a(x_i)$, $C_{e,i} = C_e(x_i)$, $x_i = x_0 + i\Delta x$ and $\Delta x = (x_N - x_0)/N$. This simplifies to

$$C_N = \tau_0^{N-1} C_0 + \sum_{i=0}^{N-1} \tau_i^{N-1} \hat{C}_{e,i} \quad (4.18)$$

where $\hat{C}_{e,i} = \sigma_{a,i} C_{e,i} \Delta x$. If $\hat{C}_{e,i}$ is interpreted as the emitted color of the fire or heated gas at location i and C_0 is interpreted as the color of the light source *behind* the smoke, then (4.18) restated in words gives color seen by the observer computed as a weighted average of source and emitted colors where each weight is the optical depth from the observer to the corresponding color location. These emitted colors can be determined from a blackbody temperature curve or from a colormap meant to show variations in temperature in terms of color.

The terms in (4.18) are summed from back to front meaning that the location of the $i = 0$ term is farthest from the observer, while the location of the $i = N - 1$ term is closest. We wish to perform this sum in reverse order, from front to back so that the sum may be terminated early if additional contributions would not significantly change the result.

Therefore, to compute C_N , let $\hat{C}_j^N$ denote the partial sum using terms $i = j$ through $i = N - 1$ in the summation term in (4.18). Using this notation $\hat{C}_N = \tau_0^{N-1} C_0 + \hat{C}_0^N$. Then

$$\hat{C}_j^N = \sum_{i=j}^{N-1} \tau_i^{N-1} \hat{C}_{e,i} \quad (4.19)$$

$$\hat{C}_{j+1}^N = \sum_{i=j+1}^{N-1} \tau_i^{N-1} \hat{C}_{e,i} \quad (4.20)$$

Subtracting (4.20) from (4.19) and solving for $\hat{C}_j^N$ results in

$$\hat{C}_j^N = \hat{C}_{j+1}^N + \tau_j^{N-1} \hat{C}_{e,j} \quad (4.21)$$

The strategy then for volume rendering an image is for each pixel in the 2D projected image to

1. convert the background radiance C_0 to a color,
2. convert the emitted radiances along the integration path to colors and
3. form a weighted average of these colors using either equation (4.18) or (4.21) where the weights are optical depths obtained using (4.16) .

The conversion from radiance to color may be based on a blackbody temperature curve if realistic flame colors are the goal or arbitrary if only a qualitative view of the fire is the goal. Equations (4.16) and (4.21) are equivalent to the recursions presented in Ref. [20, Chapter 39] for performing volume rendering.

4.2.3 Splitting the Radiation Transport Equation

This section discusses splitting the Radiation Transport Equation (RTE), so that it may be solved separately on multiple meshes or multiple slice planes. The final solution is then obtained by appropriately combining solutions obtained on each mesh or slice plane.

It is more practical to draw smoke one mesh at a time when visualizing multiple mesh cases since required data may not be easily accessible from all meshes, especially if the GPU¹ is used for drawing. The RTE solution or equivalently the computed radiance and smoke opacity require properties for the entire line of sight which may encompass more than one data mesh. This section discusses how radiance and opacity may be computed by combining solutions from each individual mesh along the line of sight. For example, as illustrated in Fig. 4.3, consider an interval $[x_0, x_N]$ that is split at D into two sub-intervals $[x_0, D]$ and $[D, x_N]$. The goal then is to compute a radiance and opacity for $[x_0, x_N]$ using radiances and opacities computed on the two sub-intervals.

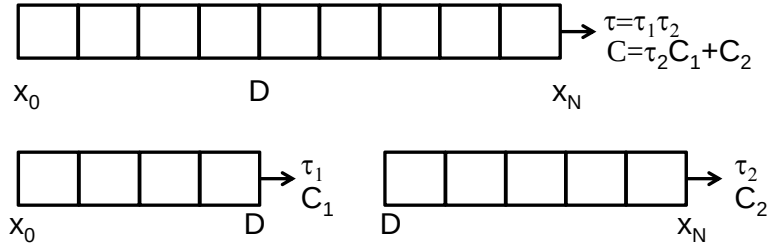


Figure 4.3: Solutions to the RTE on two sub-intervals are combined to form a solution to the RTE for the merger of these intervals.

Let C be a radiance computed on $[x_0, x_N]$ and C_1 and C_2 be radiances computed on two sub-intervals of $[x_0, x_N]$. Likewise, let τ be an optical depth computed on $[x_0, x_N]$ and τ_1 and τ_2 be optical depths computed on two sub-intervals of $[x_0, x_N]$. From (4.9) and (4.10), the radiance C and optical depth τ are given by

$$C = \int_{x_0}^{x_N} \tau(x, x_N) \sigma_a(x) C_e(x) dx \quad (4.22)$$

$$\tau = \tau(x_0, x_N) \quad (4.23)$$

Likewise, the radiances C_1 and C_2 and optical depths τ_1 and τ_2 for the intervals $[x_0, D]$ and $[D, x_N]$ are given by

¹graphics processing unit of the video card

$$C_1 = \int_{x_0}^D \tau(x, D) \sigma_a(x) C_e(x) dx \quad (4.24)$$

$$C_2 = \int_D^{x_N} \tau(x, x_N) \sigma_a(x) C_e(x) dx \quad (4.25)$$

$$\tau_1 = \tau(x_0, D) \quad (4.26)$$

$$\tau_2 = \tau(D, x_N) \quad (4.27)$$

The optical depth τ can be split into two parts giving

$$\tau = \tau(x_0, x_N) = \tau(x_0, D) \tau(D, x_N) = \tau_1 \tau_2 \quad (4.28)$$

since from (4.10) it can be shown that for any x , $\tau(a, b) = \tau(a, x) \tau(x, b)$. The radiance C can also be split into two parts giving

$$C = \int_{x_0}^{x_N} \tau(x, x_N) \sigma_a(x) C_e(x) dx \quad (4.29)$$

$$= \int_{x_0}^D \tau(x, x_N) \sigma_a(x) C_e(x) dx + \int_D^{x_N} \tau(x, x_N) \sigma_a(x) C_e(x) dx \quad (4.30)$$

$$= \tau(D, x_N) \int_{x_0}^D \tau(x, D) \sigma_a(x) C_e(x) dx + \int_D^{x_N} \tau(x, x_N) \sigma_a(x) C_e(x) dx \quad (4.31)$$

$$= \tau_2 C_1 + C_2 \quad (4.32)$$

Summarizing, full-interval values C and α may be written in terms of sub-interval values C_1 , C_2 , α_1 and α_2 using

$$\tau = \tau_1 \tau_2 \quad (4.33)$$

$$C = \tau_2 C_1 + C_2 \quad (4.34)$$

Equations (4.33) and (4.34) may then be used to draw smoke and fire one mesh at a time. These two equations are also the basis for combining RTE solutions obtained on multiple slice planes, which is discussed next.

4.3 Slice Rendering

A slice rendering algorithm for visualizing smoke consists of splitting the RTE across individual slice planes within a single mesh. The 3D computational domain is partitioned into a series of 2D slices. The RTE is then solved on each slice. Each slice solution only accounts for conditions between adjacent slices. The individual partially transparent slice solutions are then combined using video hardware to form the final image. Problems can occur with numerical round off error if two slices are too close together which require solution methods involving data volumes rather than data slices. These methods are discussed in the next section.

There are many ways to slice a 3D data set. The slice orientation is chosen to be the one most perpendicular to the viewer's line of sight, for which possible choices are slice planes parallel to the three cartesian coordinate planes (XY, XZ, YZ) or planes diagonal to the data. The opacity at each grid node is computed using the distance Δx between adjacent YZ planes and soot density data computed by the fire model. If slice orientations other than YZ are displayed, then opacities are adjusted if the distance between planes is

different than Δx . Opacity data is computed and compressed using run length encoding as a preprocessing step and decompressed one frame at a time as data is displayed.

Summarizing, a slice rendering algorithm for visualizing smoke consists of splitting the RTE across individual slice planes within a single mesh. The 3D computational domain is partitioned into a series of 2D slices. The RTE is then solved on each slice. Each slice solution only accounts for conditions between adjacent slices. The individual partially transparent slice solutions are then combined using video hardware to form the final image. Slices become more transparent as they come closer together. Problems can then occur resulting in poor visualizations because of numerical round off error. In this case, different solution techniques are required such as volume rendering which integrates the entire RTE at once rather than one slice at a time.

4.3.1 Computing Color

Smokeview visualizes smoke and fire by drawing a series of triangles in equally spaced parallel planes. Color for these triangles are assigned by mapping temperature or hrrpuv (heat release per unit volume) values to color such as with a color map illustrated in Figure 4.4. Transparency for these triangles is assigned using soot density, the greater the soot density, the more opaque the triangle.

The example color map in Fig. 4.4 is split into two parts. The left half is used for coloring non-burning regions, the right half is used for coloring burning regions. An hrrpuv cutoff value denoted $\text{hrrpuv}_{\text{cutoff}}$ is used to distinguish these two regions. If an hrrpuv value is below the cutoff, smoke is drawn using colors from the left half of the color map, while if an hrrpuv value is greater than the cutoff, fire is drawn using colors from the right half of the color map. The color map is defined as a table of 256 red, green, blue color triplets. A formula giving a color index for a given hrrpuv value is given by

$$\text{color index} = \begin{cases} 127 \frac{\text{hrr}}{\text{hrr}_{\text{cutoff}}} & 0 \leq \text{hrr} \leq \text{hrr}_{\text{cutoff}} \\ 127 + 128 \frac{\text{hrr} - \text{hrr}_{\text{cutoff}}}{\text{hrr}_{\text{max}} - \text{hrr}_{\text{cutoff}}} & \text{hrr}_{\text{cutoff}} \leq \text{hrr} \leq \text{hrr}_{\text{max}} \end{cases} \quad (4.35)$$



Figure 4.4: Example colormap used for converting temperature or hrrpuv values to color.

4.3.2 Computing Opacity

Computing opacity at slice plane nodes is illustrated in Fig. 4.5. A ray travels from the background to the observer through intervening smoke. Light is absorbed or scattered by the smoke as the ray passes each slice plane. Emission effects are accounted for by coloring the smoke. Scattering effects presently are only accounted for in the value of the total mass extinction coefficient. Light losses are assumed to be from both absorption and scattering. The obscuration is computed along each ray one grid plane at a time, using the Beer-Lambert law as follows. The $\alpha = 1 - \tau$ values are pre-computed by FDS using the Beer-Lambert law [16]

$$\alpha = 1 - \exp(-\sigma_t \Delta x) \quad (4.36)$$

for a particular view direction (down the x axis) where Δx is this distance between two grid planes and as before $\sigma_t = \sigma_a + \sigma_s$ is the total mass extinction coefficient. The Beer-Lambert law is an empirical relationship relating light absorption to the material properties of the medium the light is travelling through, in this case soot or smoke.

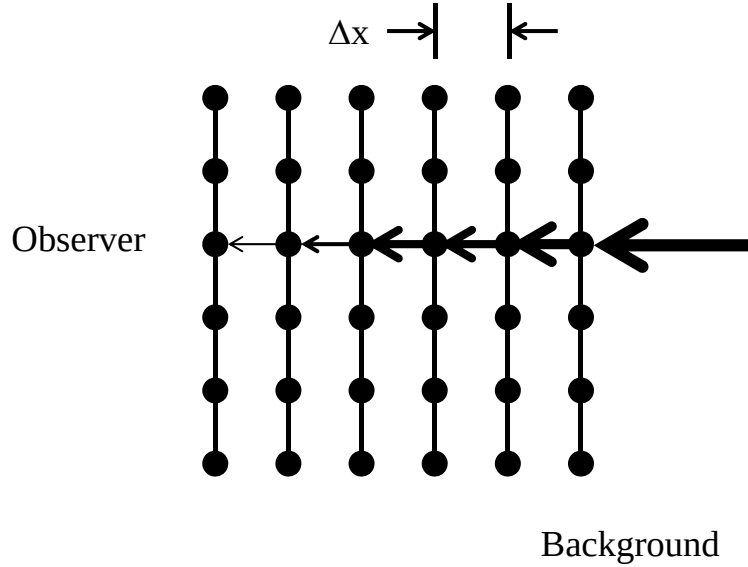


Figure 4.5: Light emitted from the background is obscured by intervening smoke.

The α parameter in (4.36) is used by OpenGL [8] to blend smoke planes with the current background. The α parameter used here also represents an opacity, 0.0, for completely transparent, and 1.0 for completely opaque.

4.3.3 Adjusting Opacity

The absorption parameter, α , needs to be adjusted for view directions not aligned with the axis orthogonal to the viewing planes (see Fig. 4.6). The absorption coefficient also needs to be adjusted when the distance between adjacent smoke planes changes, or viewing planes are skipped.

Ten million exponential operations per second are required to display smoke with corrected α 's at 10 frames per second if the simulation has grid dimensions of $100 \times 100 \times 100$. Recent advances in CPU and video hardware makes these types of visualizations possible. These corrections may also be performed in the video card (GPU), resulting in increased display rates because the GPU performs the corrections simultaneously at all or many of the grid nodes rather than one at a time as the CPU would.

The α obscurations are pre-computed using the distance Δx between adjacent planes along the x axis. The adjusted $\hat{\alpha}$ expressed in terms of $\Delta \hat{x}$ is given by

$$\hat{\alpha} = 1 - \exp(-\sigma_t \Delta \hat{x}) \quad (4.37)$$

where $\Delta \hat{x}$ is the distance between planes along the line of site. Equations (4.36) and (4.37) may be used to solve for $\hat{\alpha}$ in terms of α to obtain

$$\hat{\alpha} = 1 - (1 - \alpha)^{\Delta \hat{x} / \Delta x} \quad (4.38)$$

after noting that

$$1 - \hat{\alpha} = \exp(-\sigma_i \Delta \hat{x}) = \exp(-\sigma_i \Delta x)^{\Delta \hat{x}/\Delta x} = (1 - \alpha)^{\Delta \hat{x}/\Delta x} \quad (4.39)$$

The computation of (4.38) is expensive because the exponential is computed at each grid node for every time step. In addition, numerical cancellation may occur for small α leading to loss of significant digits. Both problems may be solved by expanding (4.38) in a Taylor series and keeping only the first few terms:

$$\hat{\alpha} \approx \alpha r - \frac{\alpha^2}{2} r(r-1) + \frac{\alpha^3}{6} r(r-1)(r-2) \quad (4.40)$$

where $r = \sec(\theta) = \Delta \hat{x}/\Delta x = ||x_p - x_e||/n \cdot (x_p - x_e)$, n is the unit vector normal to the current plane being drawn, θ is the angle between the view direction and n , x_e is the observers position and x_p is the vertex being drawn (along the view direction). These terms are illustrated in Fig. 4.6.

When planes are skipped, (4.38) may be simplified. In particular, when every 2nd plane is skipped, $\Delta \hat{x}/\Delta x = 2$, so that (4.38) simplifies to

$$\hat{\alpha} = 1 - (1 - \alpha)^2 = 2\alpha - \alpha^2 \quad (4.41)$$

The video hardware uses α values contained in the smoke planes to obscure the background much like a camera uses a neutral density filter to darken a scene. Extending the analogy, Smokeview uses one spatial/time varying *numerical* neutral density filter for each plane of smoke data. On a node by node basis then, each smoke plane obscures the current image stored in the OpenGL back buffer by the amount $(1 - \alpha)$ to form a new back buffer image. Figure 4.7 illustrates this process showing several snapshots of a fire plume. The final image in the lower right is the most realistic. A simplistic description of one step of this process is given by

$$\text{new buffer image} = (1 - \alpha) \times \text{old buffer image} \quad (4.42)$$

This process is repeated for each smoke plane.

Figure 4.8 illustrates this process showing smoke and fire in a townhouse kitchen fire. The visualization is performed by displaying a series of partially transparent planes. For illustration, these planes are made more conspicuous (in Fig. 4.8a) by skipping smoke planes (displaying every third plane) and orienting them along the x axis. Figure 4.8b shows the visualization as it normally appears with all slice planes shown and oriented along a plane most perpendicular to the view direction. Figure 4.9 shows smoke planes oriented along YZ planes but viewed down the X axis so the individual planes are not seen. The smoke planes in this figure are oriented the same as in Figure 4.8a.

4.3.4 Orienting Slice Planes

Smoke opacity data computed as described in the previous sections is stored in a 3D array. This array corresponds to the solution domain as set up in an FDS input file (or some other model). Smoke planes are drawn in Smokeview through this data. The orientation is chosen to be most perpendicular to the viewer's

line of sight. A plane orientation exactly perpendicular to the view direction could be drawn if one is willing to pay the added CPU cost of interpolating opacity values between grid nodes.

Figure 4.10 illustrates this process showing three view directions and the corresponding smoke plane orientations that would be used. Off-axis viewing is minimized by selecting the view planes orientation that minimizes the angle between the planes normal direction and the view direction. This angle, θ , is illustrated in Fig. 4.11, and is given by

$$\cos(\theta) = \frac{n \cdot v_e}{||n|| ||v_e||} \quad (4.43)$$

where n is normal vector for the candidate smoke plane, and v_e is the view direction vector. In OpenGL, the view direction vector, v_e , is computed by simply obtaining the OpenGL modelview matrix, M and multiplying it by the vector, $(0, 0, 1)^T$ or equivalently the third row of M .

4.3.5 Compressing Slice Plane Data

The opacity parameters are computed at each slice plane node for all time steps. The space required to store these values can easily become quite large. Compression techniques are required to reduce storage requirements.

Storage reduction occurs in two steps. First, four byte floating point soot densities are converted to one byte smoke opacities using the Beer-Lambert law. Video cards presently use only one byte to represent opacity. Next, the sequence of opacity values are compressed using run-length encoding, a compression scheme where repeated “runs” of data are replaced with a number (number of repeats), and the value repeated. In more detail,

1. Represent four or more consecutive identical characters as $\#nc$ where $\#$ is a special character denoting the beginning of a repeated sequence, n is the number of repeats and c is the character repeated. n can be up to 254 (255 is used to represent the *special* character).
2. Represent characters not repeated four or more times as is.

The character string `aaaaaabbcc` would then be encoded as `#6a#4bcc`.

Run length encoding provides a reasonably good compression ratio, is simple to implement and more importantly can be decompressed quickly. This last property is important for any compression scheme chosen because it is a rate limiting step in the process that Smokeview uses to display smoke data. The CPU time required to compute the smoke flow can easily exceed one minute of CPU time per output time step, so extra time used to produce a more compact file is affordable. However, each data frame is decompressed *on the fly* so a compression format that can be rapidly decompressed is critical.

A second compression scheme is used by Smokezip, companion software to FDS and Smokeview, to more compactly compress FDS files. Smokezip uses the ZLIB compression library [21].

4.3.6 A Limitation of the Slice Solution Method

As noted earlier, the slice rendering method for visualizing smoke records opacities on grid planes using

$$\alpha = 1 - \exp(-\sigma_t \Delta x) \quad (4.44)$$

where again Δx is the distance between adjacent grid planes. Problems with round off error can occur because α 's are stored using only 8 bits, the size typical video hardware uses to represent color and alpha

channels. As a result, opacity values are quantized. Only values of 0, $1/255$, $\dots$, $254/255$ and 1 can be represented. In particular, any α value computed to be smaller than $1/255$ will truncate to 0. This can occur when more grid cells are used to resolve a solution domain, *i.e.* when Δx is sufficiently small so that $\alpha < 1/255$.

For example, suppose that a 1 m column of smoke has opacity of 0.5. Substituting $\Delta x = 1$ and $\alpha = 1/2$ into (4.44) gives $\sigma_i = -\ln(1/2)$. Solving for Δx using this value of σ_i and $\alpha = 1/255$ gives

$$\Delta x = \frac{\ln(254/255)}{\ln(1/2)} \quad (4.45)$$

Therefore, the opacity, α , will truncate to zero (for this example) whenever $\Delta x < \ln(254/255)/\ln(1/2) \approx 0.00567$. Equivalently, the opacity truncates to zero when $N > \ln(1/2)/\ln(254/255) \approx 177$, where N is the number of grid planes in the 1 m column of smoke. Smoke drawn in this situation will be *invisible*.

To overcome this problem, the RTE needs to be solved across the full 3D data mesh where intermediate smoke opacities are stored using full precision arithmetic. One such algorithm is discussed in the next section.

4.4 Volume Rendering

Volume rendering is the process of visualizing 3D data by projecting partially transparent colors derived from 3D data onto a 2D plane forming an image. An entire volume of data is used to generate an image rather than just a slice as in the previous section. Transfer functions or colormaps are used to map data to color and optical density (opacity). In this application temperature is mapped to color and soot density is mapped to opacity. These colors and opacities are then combined to form an image. The transfer functions may be arbitrarily designed to highlight certain portions of the data or based on physics designed to produce realistic appearing images. Figure 4.12 illustrates this process. Soot density is mapped to opacity. The colors and opacities are then combined to form an image. The strategy then is to perform color mixing in the same way that light would behave by solving an approximate form of the radiation transport equation. The approximate RTE solution given in (4.9) is used to collapse 3D data into 2D images projected onto the sides of the mesh visible to the observer.

Figure 4.13 illustrates the effect of projecting an image onto a plane. The top center image is presented from the point of view of the observer. The perspective is correct. The two images below show the same image but from a different viewpoint. The scene is rotated left and right to show more clearly the images projected on the front facing sides of the data mesh. As the scene is moved through rotations and translations the projected surface images are constantly recomputed and redrawn presenting the illusion that the image is 3D and drawn within (rather than on the surface) of the data mesh.

4.4.1 Implementation

Data required to volume render smoke is made available to Smokeview by adding lines to an FDS input file specifying 3D slice files for temperature and soot density such as

```
&SLCF XB=xmin,xmax,ymin,ymax,zmin,zmax, QUANTITY='TEMPERATURE' /
&SLCF XB=xmin,xmax,ymin,ymax,zmin,zmax, QUANTITY='DENSITY', SPEC_ID='SOOT' /
```

where xmin, $\dots$, xmax represent the domain boundary.

An algorithm for determining image opacities and colors using volume rendering is detailed below.

1. For each pixel in the image plane, cast a ray from the observer's viewpoint through that pixel into the 3D data set
2. Step along the ray from the front (relative to the observer) to the back of the 3D data set converting data values along the way to color and opacity. Choose a step size (possibly varying) to capture changes occurring in the data set.
3. Combine the colors and opacities found in step 2 using recursions equations (4.16) and (4.21). Initiate the recursion with $\hat{\alpha}_N = \hat{C}_N = 0$. Continue the recursion for $i = N - 1$ to $i = 0$ by computing $\hat{\alpha}_i$ and $\hat{C}_i$ using:

$$\hat{\alpha}_i = \hat{\alpha}_{i+1} + (1 - \hat{\alpha}_{i+1}) \alpha_i \quad (4.46)$$

$$\hat{C}_i = \hat{C}_{i+1} + (1 - \hat{\alpha}_{i+1}) C_i \quad (4.47)$$

where $\hat{C}_i$ and $\hat{\alpha}_i$ are color and opacity accumulated from steps N to i while stepping through the 3D data set from front to back. When data is mapped appropriately to color and opacity, this recursion is simply a numerical integration of the radiation transport equation. The values C_i and α_i are the color and opacity at the i 'th step.

4. Mix the volume rendered color, $\hat{C}_N$, with the colors already rendered using equation (4.34) re-written as

$$\text{updated background color} = (1 - \hat{\alpha}_N) \times \text{original background color} + 1 \times \hat{C}_N \quad (4.48)$$

The OpenGL call, `glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA);`, is used to implement this mixing mode in Smokeview.

A limitation of the volume rendering procedure is the large file sizes required to store the full precision being used in computations. For example, smoke flow computed on a $128 \times 128 \times 128$ mesh for 1024 time steps requires 16 gigabytes to store data for visualization. As with the slice rendering method, compression procedures are implemented using the ZLIB library [21] to reduce the file size.

4.5 Future Work

This chapter describes the algorithms Smokeview uses to display smoke and fire using physics based algorithms. These algorithms may be improved in several ways. Presently, only radiation from soot is used to visualize smoke. The gray gas assumption may be relaxed by solving the RTE for several wavelength bands (as may be done by FDS) and combining the results. The RTE line integration needs to terminate at the first solid object encountered (FDS OBST) rather than the far side of the data mesh. The computational efficiency may be improved by implementing algorithms to skip over regions with little or no smoke. Research on unstructured geometries for future incorporation into FDS may also lead to better visualization. Finally, flame color computations may be made more quantitative by using a transfer function relating temperature to color that is physics based rather than an assumed color map.

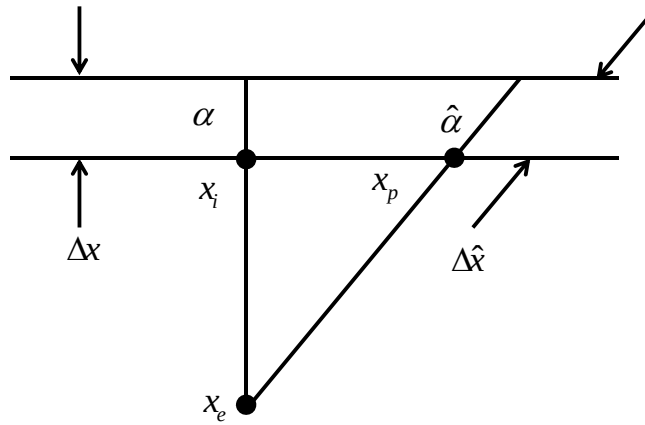


Figure 4.6: Diagram illustrating the adjustment required to the opacity parameter, α , for non axis aligned views. The adjusted opacity $\hat{\alpha}$ along the $\Delta\hat{x}$ segment is related to α value along the Δx segment using $(1 - \hat{\alpha}) = (1 - \alpha)^{\Delta\hat{x}/\Delta x}$

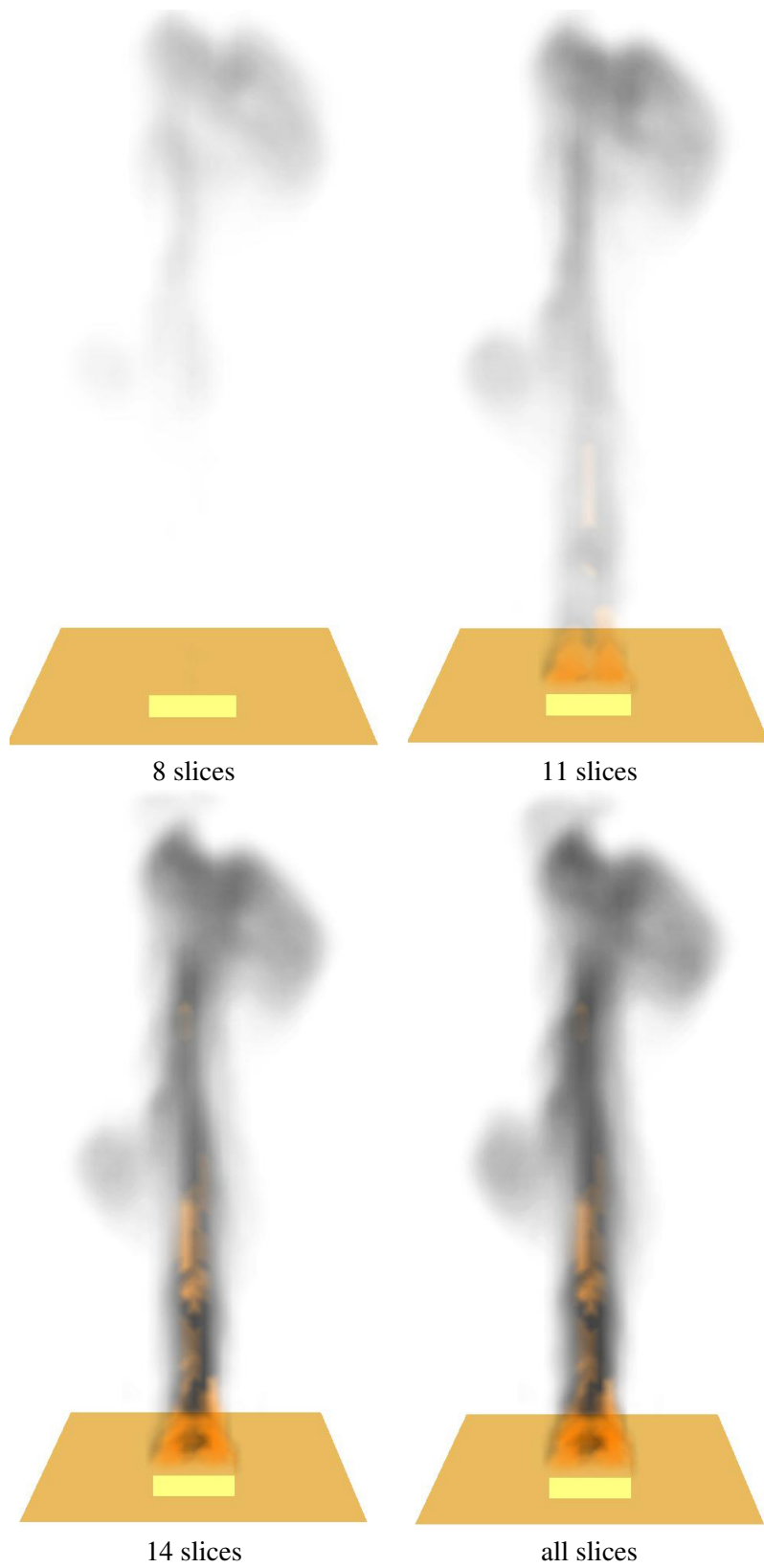
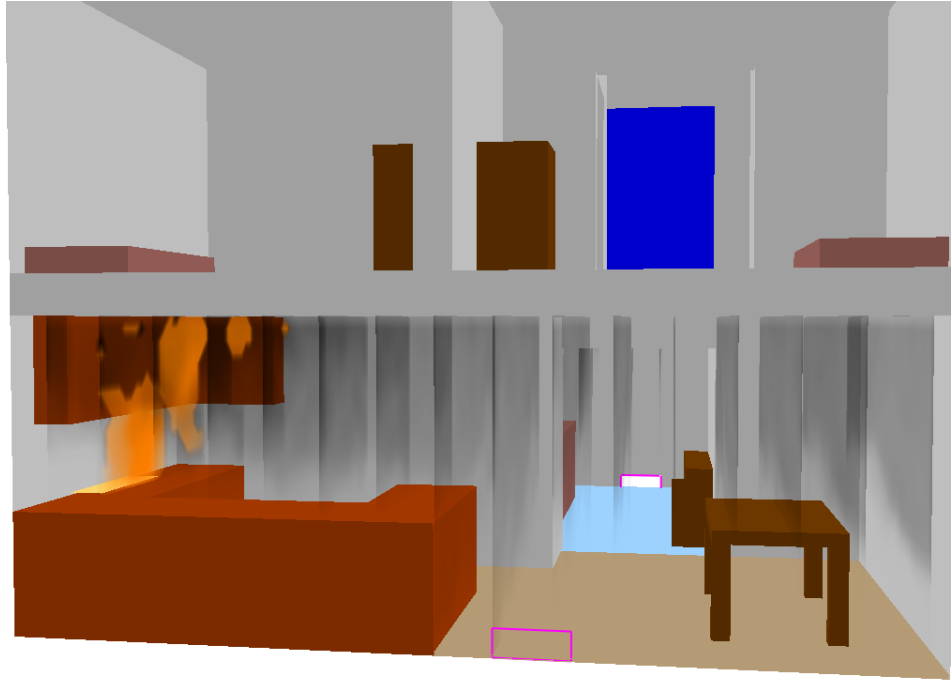
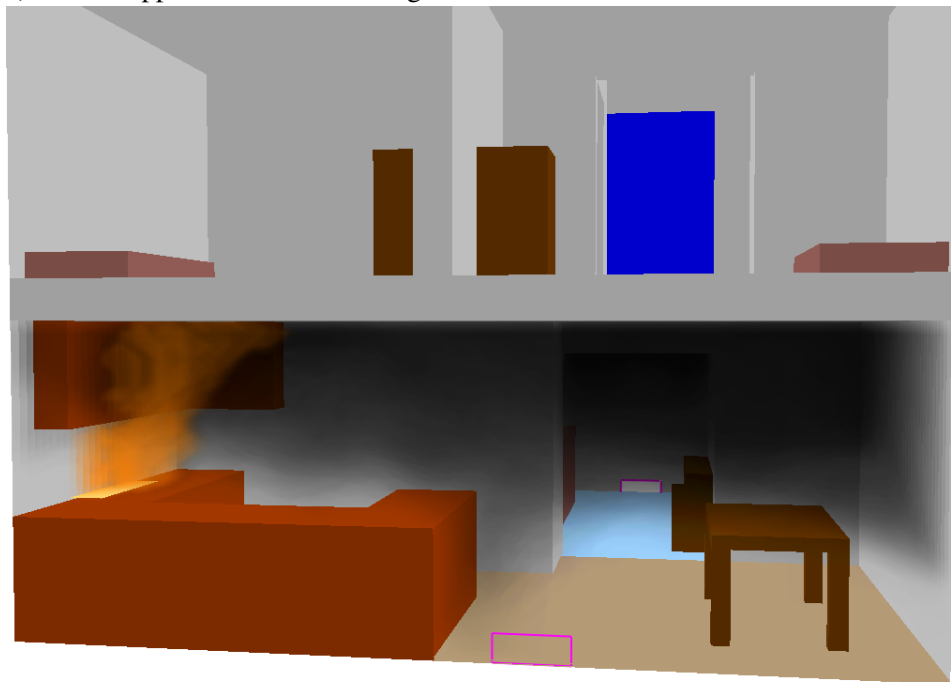


Figure 4.7: Smoke plume visualized using several vertical parallel partially transparent planes. The smoke plume looks more realistic as more slice planes are included to form the image.



a) slices skipped and oriented along the 'X' directions



b) all slices shown and oriented towards the viewer

Figure 4.8: Realistic visualization of a townhouse kitchen fire simulated using FDS. For illustrative purposes, planes in the top image are oriented along the x axis. Planes in the bottom image are aligned along the y axis, the axis most perpendicular to the line of sight.

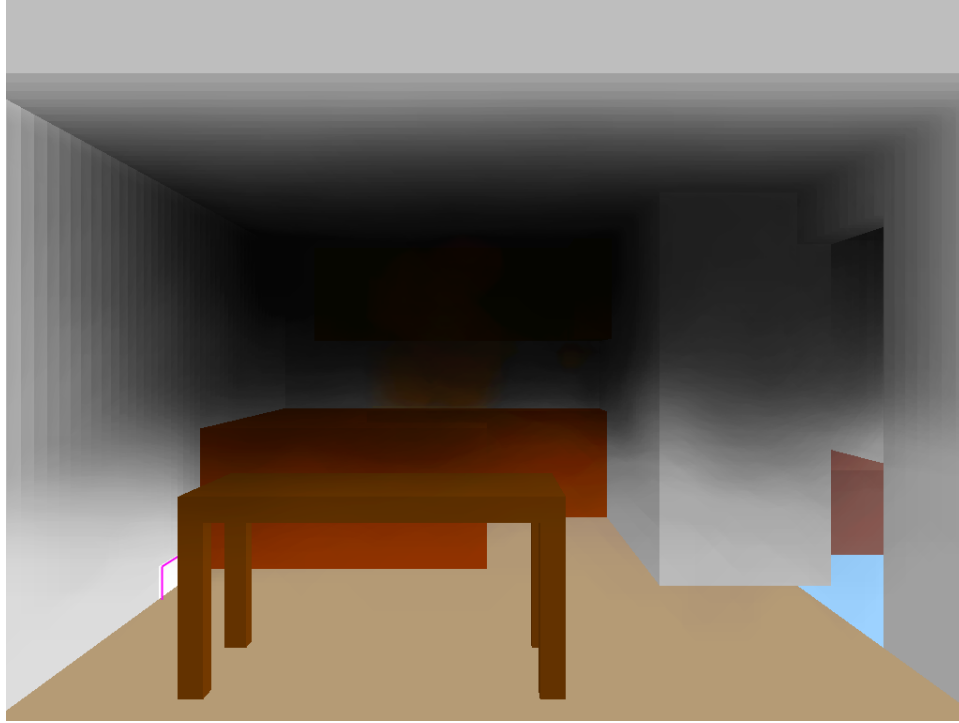


Figure 4.9: Realistic visualization of a townhouse kitchen fire simulated using FDS viewed along the X axis.

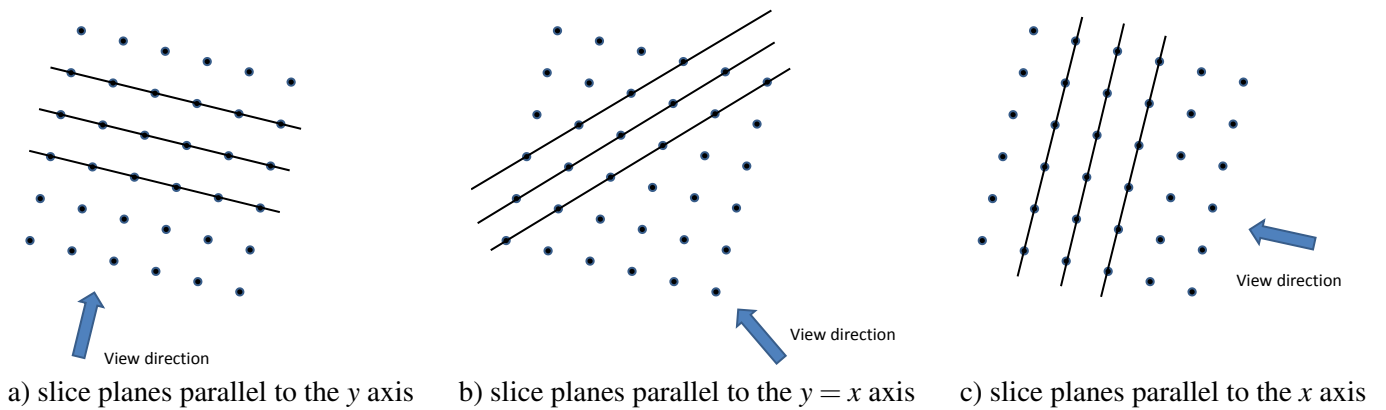


Figure 4.10: Slice plane orientation chosen to be *most perpendicular* to the line of sight.

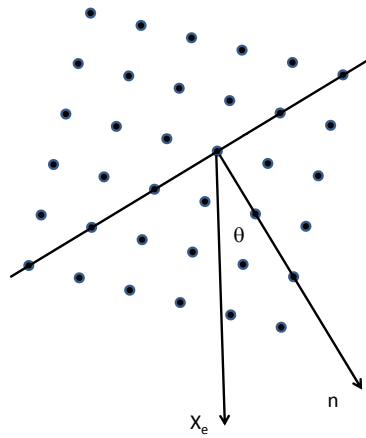


Figure 4.11: Diagram illustrating the angle between the line of sight and the vector normal to the slice planes. Slice plane orientation is chosen to minimize this angle.

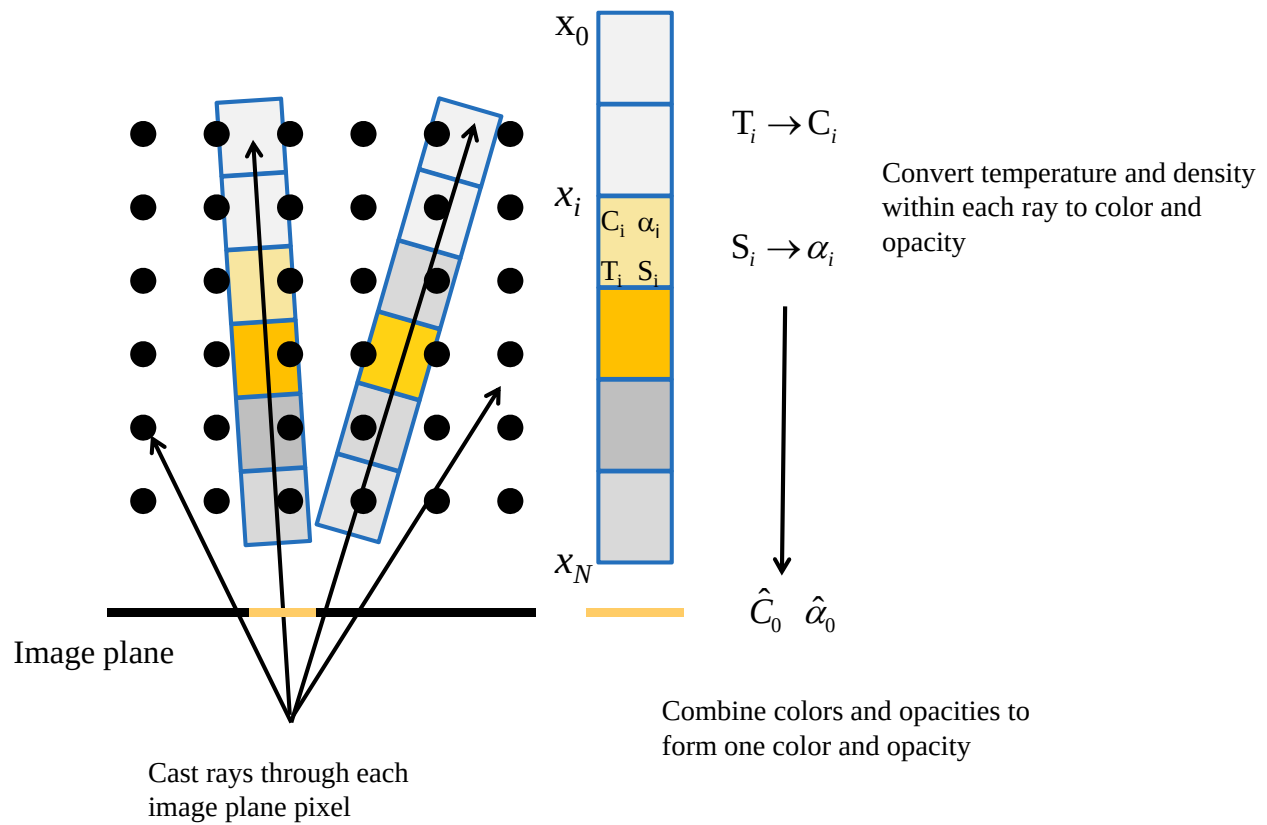
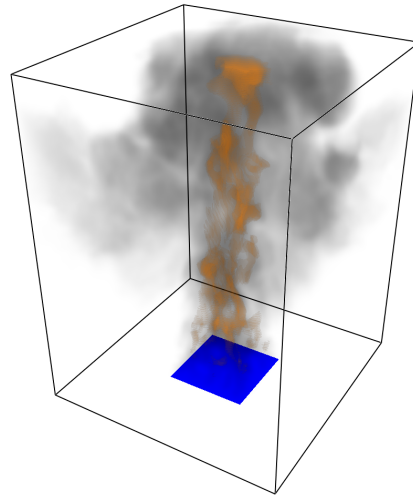
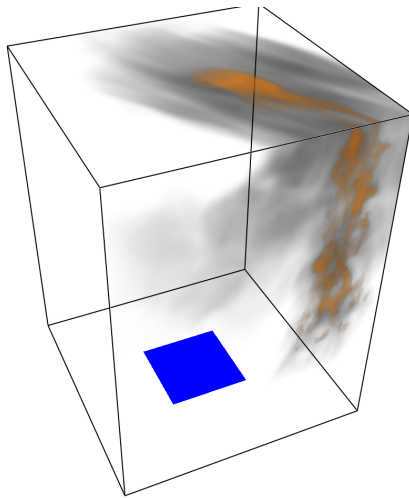


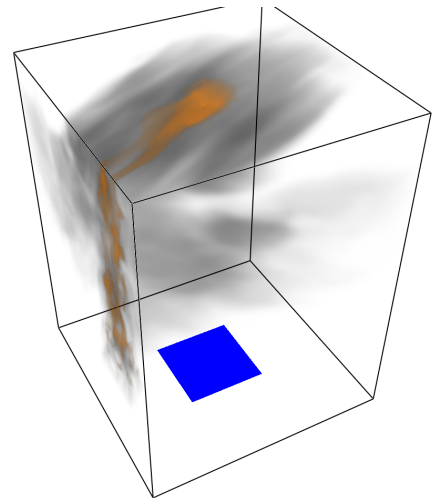
Figure 4.12: Opacity and color is computed for each pixel on an image plane by solving a line integral representation of a simplified form of the radiation transport equation. Rays are cast from the observer through the image plane into the 3D data set converting temperatures to color and soot densities to opacity. The line integral is computed for each pixel where the ray intersects the data set.



a) normal view as seen by the observer



b) same image as in a) but viewed from the left



c) same image as in a) but viewed from the right

Figure 4.13: Volume rendered smoke plume projected onto the outside surfaces of the data mesh and shown from several points of view. The image, in a), is as viewed by the observer. The other two images, b) and c), are rotated versions of the image as in a).

Chapter 5

Future Work

Smokeyview is a software tool used to gain insight into results generated by fire models such as the Fire Dynamics Simulator or CFAST. Two general areas of research need to be addressed in order to improve this tool. First, scenarios with a large number of grid cells need to be visualized more effectively and efficiently and second, some visualization algorithms need to be improved and others need to be added in order to more effectively interpret fire modeling data. Some areas of research to accomplish these goals are described in more detail in the following sections.

5.1 Visualize Cases more Realistically

Realism is a metric used to gauge qualitatively the accuracy of fire and smoke display. Realism itself is not the primary goal, however. It is a side effect of the application of more physics. The primary goal is to gain a better understanding of the data. This requires a more accurate representation of the underlying data which in turn requires the application of more physics. Two areas being investigated are

- Investigate techniques for visualizing fire more realistically by using information about the fire such as its temperature and composition (soot and various gas species) to color it more realistically. Flame temperatures need to be modeled directly by FDS in order to use the blackbody temperature curve to obtain flame color. Further, the resolution of the computation needs to be consistent with the desired resolution of the flame image.
- Investigate algorithms for modeling the interaction of light and smoke. The *transport* of light into and out of the smoke/fire (*i.e.*, *scattering*) is another important factor that effects its appearance. Light sources could consist of either man made lights or from the fire itself. Work also needs to be done to generalize the wavelength of light used to visualize the scene, *i.e.*, implementing an algorithm in Smokeyview to simulate a thermal imager or infrared viewer.

5.2 Fire Computations in Smokeyview

The roles of FDS and Smokeyview seem clear, FDS performs fire computations and Smokeyview visualizes them. This distinction became blurred with the addition of 3D smoke visualization algorithms in Smokeyview. Further blurring the distinction, pre-visualization steps are performed in FDS, to convert soot density to a smoke opacity. These two codes represent a suite of software dedicated to advanced scientific fire modeling and visualization.

Additional fire computations are planned for Smokeview. The user will gain insight into the fire phenomena much more quickly by being able to manipulate and solve their problem in real time. Two examples are detailed below.

- Investigate methods for modeling the motion of fire brands using either wind fields computed by FDS or wind fields defined in Smokeview. This allows one to define the initial position and distribution of fire brands and to note where they land.
- Implement a level set method proposed by Rehm and McDermott [22] for tracking an outdoor fire line. The method would take into account terrain (i.e., non-level ground) defined by the user for outdoor fire applications.

5.3 Visualize Larger Cases

Techniques need to be investigated for visualizing larger cases more effectively. Updating Smokeview to allow 64 bit memory addressing is just one option. This is a brute force method. Techniques are also required for honing or zeroing in on data of interest. The fire line used to visualize WUI (wildland urban interface) simulations is a good example of this. A fire line in the context of Smokeview is simply a visual display of temperature only visible where the fire is located. Additional techniques for visualizing large cases that need to be investigated include:

- Investigate methods for making it easier to probe or mine data, i.e., to retrieve data of particular interest. Now FDS data is stored sequentially. For large data sets one needs to retrieve data efficiently at a particular time and region without reading the entire data set.

User defined spatial and temporal data subsets need to be loaded efficiently in order to shorten the time required to visualize cases.

- Investigate standardized methods for storing data more efficiently and more effectively. Can we do it better and be practical? The idea would be not to change how FDS outputs data but to consider whether a separate *filter* program that would convert FDS generated data into a different format that would enable other tools besides Smokeview to be able to visualize data, for example the visualization tool kit VTK [23].
- Investigate parallelization methods for visualizing data. Smokeview presently has a limited ability to execute code in parallel using a technique known as multi-threading. For example, smooth blockages if present, are smoothed in parallel using the pthreads library. Smokeview in this respect is multi-threaded. Techniques will be investigated for performing the drawing or visualization in parallel. This may be necessary as cases get larger and larger. One technique for parallelizing the visualization is to use several video cards each one drawing a portion of the scene.

5.4 Tools and Techniques

- Investigate methods for using the video card to perform scientific computations. The computational power of the video card is already being exploited by Smokeview to perform simple smoke computations. It will need to be exploited even more in order to make the proposed fire coloring and smoke lighting computations practical. Techniques are being investigated for performing the computations needed to implement the more realistic fire and smoke computations discussed earlier, in particular CUDA [24]. Techniques such as these will be required if more complex visualization algorithms are to be effective.

- Investigate alternative methods for implementing a graphical user interfaces (GUI) for Smokeview. Smokeview uses GLUT for implementing a simple user interface. GLUT was not intended for developing user interfaces as complex as what Smokeview requires. Alternatives for implementing user interfaces will be investigated.
- Currently FDS and Smokeview run in batch mode without using updated information. As FDS or CFAST begin to use data assimilation techniques, it would be useful if Smokeview could incorporate updated results as they are computed.

References

- ¹G. Forney, *Smokeyview, A Tool for Visualizing Fire Dynamics Simulation Data, Volume I: User's Guide*, NIST Special Publication 1017-1 (National Institute of Standards and Technology, Gaithersburg, Maryland, USA, May 2013), [10.6028/NIST.sp.1017-1](https://doi.org/10.6028/NIST.sp.1017-1) (cit. on pp. i, 1, 41).
- ²G. Forney, *Smokeyview, A Tool for Visualizing Fire Dynamics Simulation Data, Volume II: Technical Reference Guide*, NIST Special Publication 1017-2 (National Institute of Standards and Technology, Gaithersburg, Maryland, USA, May 2013), [10.6028/NIST.sp.1017-2](https://doi.org/10.6028/NIST.sp.1017-2) (cit. on p. i).
- ³G. Forney, *Smokeyview, A Tool for Visualizing Fire Dynamics Simulation Data, Volume III: Verification Guide*, NIST Special Publication 1017-3 (National Institute of Standards and Technology, Gaithersburg, Maryland, USA, May 2013), [10.6028/NIST.sp.1017-3](https://doi.org/10.6028/NIST.sp.1017-3) (cit. on pp. i, 1, 3, 28).
- ⁴K. McGrattan, S. Hostikka, R. McDermott, J. Floyd, C. Weinschenk, and K. Overholt, *Fire Dynamics Simulator, User's Guide*, NIST Special Publication 1019 (National Institute of Standards and Technology, Gaithersburg, Maryland, USA, Sept. 2013), [10.6028/NIST.sp.1019](https://doi.org/10.6028/NIST.sp.1019) (cit. on pp. i, 1, 2).
- ⁵K. McGrattan, S. Hostikka, R. McDermott, J. Floyd, C. Weinschenk, and K. Overholt, *Fire Dynamics Simulator, Technical Reference Guide, Volume I: Mathematical Model*, NIST Special Publication 1018-1 (National Institute of Standards and Technology, Gaithersburg, Maryland, USA, Sept. 2013), [10.6028/NIST.sp.1018](https://doi.org/10.6028/NIST.sp.1018) (cit. on p. i).
- ⁶K. McGrattan, S. Hostikka, R. McDermott, J. Floyd, C. Weinschenk, and K. Overholt, *Fire Dynamics Simulator, Technical Reference Guide*, NIST Special Publication 1018, Vol. 1: Mathematical Model; Vol. 2: Verification Guide; Vol. 3: Validation Guide; Vol. 4: Software Quality Assurance (National Institute of Standards and Technology, Gaithersburg, Maryland, USA, Sept. 2013), [10.6028/NIST.sp.1018](https://doi.org/10.6028/NIST.sp.1018) (cit. on p. 1).
- ⁷R. Peacock, K. McGrattan, G. Forney, and P. Reneke, *CFAST – Consolidated Fire And Smoke Transport (Version 7) Volume 1: Technical Reference Guide*, Technical Note 1889v1 (National Institute of Standards and Technology, Gaithersburg, Maryland, Nov. 2015), [10.6028/NIST.TN.1889v1](https://doi.org/10.6028/NIST.TN.1889v1) (cit. on p. 1).
- ⁸D. Shreiner, G. Sellers, J. M. Kessenich, and B. Licea-Kane, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3*, 8th ed. (Addison-Wesley Professional, Upper Saddle River, NJ, 2013) (cit. on pp. 1, 3, 49, 67).
- ⁹M. J. Kilgard, *OpenGL Programming for the X Window System* (Addison-Wesley Developers Press, Reading, Massachusetts, 1996) (cit. on pp. 1, 67).
- ¹⁰B. W. Kernighan and D. M. Ritchie, *C Programming Language (2nd Edition)* (Prentice Hall PTR, Mar. 1988) (cit. on p. 1).
- ¹¹T. M. Ellis, I. R. Philips, and T. M. Lahey, *Fortran 90 Programming* (Addison-Wesley, 1994) (cit. on p. 1).
- ¹²P. Martz, *OpenGL Distilled*, 1st ed. (Addison-Wesley, Upper Saddle River, NJ, 2006) (cit. on p. 3).

- ¹³G. Sellers, R. S. W. Jr., and N. Haemel, *OpenGL SuperBible: Comprehensive Tutorial and Reference*, 7th ed. (Addison-Wesley, Upper Saddle River, NJ, 2015) (cit. on p. 3).
- ¹⁴W. E. Lorensen and H. E. Cline, “Marching cubes: A high resolution 3D surface construction algorithm”, [Proceedings of the 14th annual conference on Computer graphics and interactive techniques](#) (1987), pp. 163–169 (cit. on pp. 19, 28).
- ¹⁵D. Purser, “SFPE Handbook of Fire Protection Engineering”, 5th ed. (Springer, New York, 2016) Chap. Combustion Toxicity (cit. on p. 34).
- ¹⁶J. R. Howell, M. P. Mengüç, K. Daun, and R. Siegel, *Thermal Radiation Heat Transfer*, 7th ed. (CRC Press, Taylor & Francis Group, New York, NY, 2021) (cit. on pp. 41, 42, 48).
- ¹⁷M. Levoy, “Display of Surfaces from Volume Data”, [IEEE Comput. Graph. Appl.](#) **8**, 29–37 (1988) (cit. on p. 41).
- ¹⁸K. Engel, M. Hadwiger, J. M. Kniss, C. Rezk-Salama, and D. Weiskopf, *Real-Time Volume Graphics* (A. K. Peters, Wellesley, Massachusetts, 2006) (cit. on p. 41).
- ¹⁹P. Dutre, K. Bala, and P. Bekaert, *Advanced Global Illumination* (A. K. Peters, Ltd., Natick, MA, USA, 2002) (cit. on p. 42).
- ²⁰R. Fernando, *GPU Gems: Programming Techniques, Tips and Tricks for Real-Time Graphics* (Pearson Higher Education, 2004) (cit. on p. 46).
- ²¹J.-I. Gailly and M. Adler, *zlib version 1.1.4*, (Nov. 2002) <https://zlib.net/> (cit. on pp. 51, 53, 67).
- ²²R. Rehm and R. McDermott, *Fire-Front Propagation Using the Level Set Method*, NISTTN 1611 (National Institute of Standards and Technology, Gaithersburg, Maryland, Apr. 2009) (cit. on p. 62).
- ²³W. Schroeder, K. Martin, and B. Lorensen, *The Visualization Toolkit, Third Edition* (Kitware Inc., Clifton Park, NY, 2004) (cit. on p. 62).
- ²⁴<https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (cit. on p. 62).
- ²⁵P. Rademacher, *GLUI version 2.1*, <https://sourceforge.net/projects/glui/files/Documentation/> (cit. on p. 67).
- ²⁶T. Boutell, (Nov. 2002) <https://libgd.github.io/> (cit. on pp. 67, 73).
- ²⁷G. E. Schalnatz, A. Dilger, and G. Randers-Pehrson, (Nov. 2002) <https://www.libpng.org/pub/png/> (cit. on p. 67).
- ²⁸*JPEG version 6b*, <https://www.ijg.org/> (cit. on p. 67).

Appendix A

Smokeyview Program Structure

This chapter gives an overview of the program structure for Smokeyview. Smokeyview consists of about 175 000 lines of code. Most of it is written in C using standard libraries such as OpenGL [8] for implementing the graphics, GLUT [9] for providing a simple menu based user interface and interacting with the host operating system. Additional graphical user interface (GUI) elements are implemented as dialog boxes using the OpenGL based widget library GLUI [25]. The libraries GD [26], libpng [27], and libjpeg [28] are used for converting Smokeyview scenes/results into images files. The library libzip [21] is used for compression. Smokeyview uses this compression library when generating PNG files and when reading in 3D smoke files compressed by Smokezip.

A small portion of Smokeyview is written in Fortran 90 to input data generated by FDS. The use of portable libraries allows Smokeyview to run on many platforms including Windows, and various versions of Unix such as IRIX (for the SGI), Linux and OS X (for the Macintosh).

Figure A.1 illustrates how these libraries and Smokeyview are organized.

Since Smokeyview is a large program and is constantly changing, it is not practical to give a line by line detailed description. The interested user, however, should be able to use this overview as a starting point to dig into those parts of Smokeyview that may be of interest.

The program structure of Smokeyview is fundamentally different than that of FDS in one important respect. Smokeyview is an *event driven* program with a graphical user interface. The term *event driven* in this context means that the user controls the program flow using various means such as pressing a key, clicking or dragging the mouse, selecting a menu item, etc. In other words, the user initiates events not the program. The Smokeyview response then depends on which *event* has occurred.

The program structure may then be described as follows. Smokeyview starts out by performing initializations and setting up the OpenGL environment. This is followed by a call to a GLUT routine that implements the *event loop*. The *event loop* is literally a program loop without an exit which detects when the user invokes the various events, again these are key clicks, mouse clicks, menu selection, etc. Once an event is detected, a user *callback* routine is called. Note that user callback routines are Smokeyview code and the event loop is GLUT library code. Part of Smokeyview's initialization process is to define which C procedures are associated with which callbacks. This is performed in the Smokeyview C routine `InitOpenGL` located in the source file `startup.c`.

There is a Smokeyview callback routine for every action that is performed. For example, the user callback for loading a 3D smoke file is `Load3DMenu`. This is a menu callback because it is called after the user selects a menu item. All menu callbacks are located in the source file `menu.h`. All other callback routines are located in the source file `callback.c`.

Figure A.2 illustrates the program structure described in the previous paragraphs. Every OpenGL/GLUT

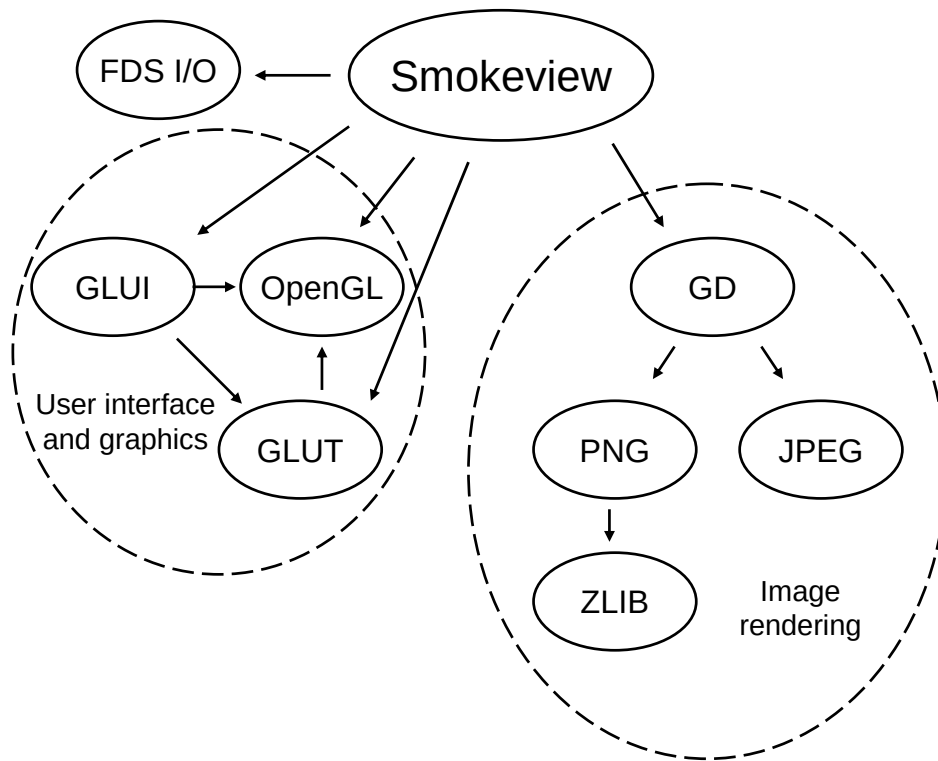


Figure A.1: Smokeview external library usage.

program will use most of the callbacks named in this figure.

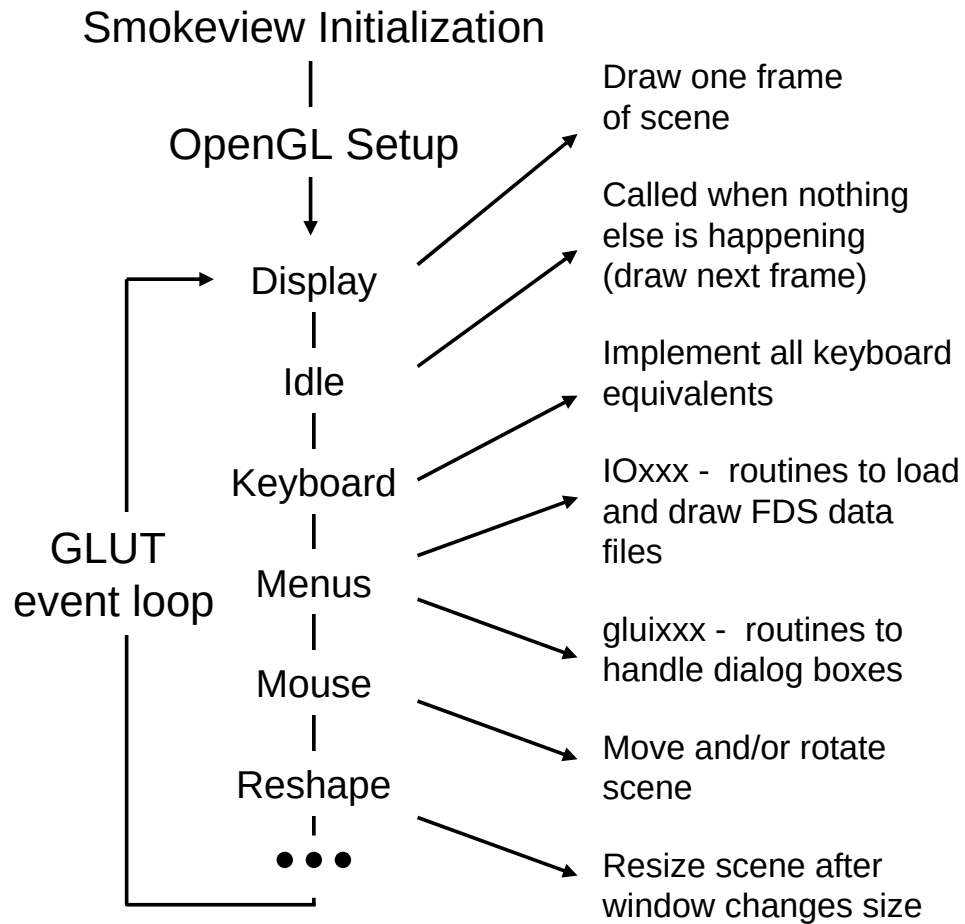


Figure A.2: Smokeview program structure.

Appendix B

Interfacing OpenGL with the Host Operating System

OpenGL by design draws the 3D geometry but does not interact with the user or the operating system. Smokeview uses the graphics library utility toolkit (GLUT) for interacting with the user *via* the keyboard and mouse and for interacting with the operating system to swap display buffers, to display fonts, to set maximum frame rates, etc. Though not as sophisticated as other libraries, GLUT is simple to use and is portable allowing Smokeview to be built on a number of different computer platforms including a PC running Windows or Linux, a Silicon Graphics workstation running IRIX or a Macintosh running OS X.

B.1 Buffers

Smokeview uses several buffers provided by OpenGL for visualization. GLUT is used to manipulate these buffers. Smokeview uses double buffering. Double buffering is the technique where drawing occurs in the **back buffer** while the scene is simultaneously displayed using the **front buffer**. Smokeview uses the GLUT routine `glutSwapBuffers()` to swap the front and back buffers once drawing is complete. Screen flickering occurs if the display (front) buffer is updated while drawing occurs.

Hidden lines and surfaces are removed from a scene using the **depth buffer**. Each time Smokeview draws an object, its depth (distance from the observer) is compared to the value previously stored in the depth buffer. If the object's depth is less than the value stored in the depth buffer then the object is considered visible and the new depth value is stored in the buffer. Otherwise the depth buffer remains unchanged and the object is considered hidden.

B.2 Initialization and Callback Routines

Initializations are performed by GLUT to set up windows and to define display modes. Smokeview defines the display to handle color, a depth buffer and double buffering by passing the OpenGL keywords `GLUT_RGB`, `GLUT_DEPTH` and `GLUT_DOUBLE` to `glutInitDisplayMode` as in

```
glutInitDisplayMode(GLUT_RGB|GLUT_DEPTH|GLUT_DOUBLE);
```

Smokeview creates a window with width `windW` and height `windH` using the GLUT calls

```
glutInitWindowSize(windW, windH);  
glutCreateWindow("");
```

and defines callbacks with

```
glutSpecialUpFunc (specialkeyboard_up);  
glutKeyboardUpFunc (keyboard_up);  
glutKeyboardFunc (keyboard);  
glutMouseFunc (mouse);  
glutSpecialFunc (specialkeyboard);  
glutMotionFunc (motion);  
glutReshapeFunc (Reshape);  
glutDisplayFunc (Display);
```

A callback is a routine that is called when a particular event occurs. For Smokeview these events would be when a keyboard key is depressed (or when the key is released), when the mouse is clicked or when the mouse is moved. Smokeview determines rotation and translation amounts using the `motion` callback defined with `glutMotionFunc (motion)`.

Appendix C

Rendering Smokeview Images

Smokeview uses the library GD [26] to convert the currently displayed scene into either a JPEG, PNG or GIF image. Smokeview reads in the OpenGL back buffer and then uses the GD routine, `gdImageSetPixel` to store the image data in GD's internal format one pixel at a time. Finally, Smokeview uses a GD routine to convert the image into the desired image format.

A summary of the steps in more detail are:

1. Allocate memory buffers and file pointers

```
RENDERfile = fopen(RENDERfilename, "wb");
pixels = (GLubyte *) malloc(width * height * sizeof(GLubyte) * 3);
```

2. Read pixel data from the back buffer

```
glReadPixels(x, y, width, height, GL_RGB, GL_UNSIGNED_BYTE, OpenGLImage);
```

3. Allocate the gd data structures used to hold image data

```
RENDERimage = gdImageCreateTrueColor(width,height);
```

4. Set pixel data

```
for (i = height-1 ; i>=0; i--) {
    for (j=0; j<width; j++) {
        r=*p++; g=*p++; b=*p++;
        rgb = (r<<16) | (g<<8) | b;
        gdImageSetPixel(RENDERimage, j, i, rgb);
    }
}
```

5. Write out data to a JPEG image file

```
gdImageJpeg(RENDERimage, RENDERfile, -1);
```

6. deallocate memory buffers and free file pointer

```
fclose (RENDERfile);  
gdImageDestroy (RENDERimage);  
free (OpenGLimage);
```